UNIT-2

Frontend Frameworks

Responsive Web Design

What is Responsive Web Design?

Responsive web design **makes** web pages render correctly on various device screen sizes without cutting short or distorting the content.

Responsive web design **makes** your web page look appropriate, suitable, and well-placed on all devices (desktop, tablet, smartphone, etc.)

Responsive web design **uses** HTML and CSS to resize, hide, shrink, enlarge, or move the content. It makes the content look good on any screen.

How to Make Your Site Responsive?

Integrating responsive web design approaches into a project includes every part of the code – within the HTML **<head>** element, the HTML structure, the CSS, and even the JavaScript.

A. Set the Viewport in the Head

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Let's break down the attributes in the meta tag:

name="viewport": This attribute specifies that the meta tag is related to the viewport settings of the webpage.

content="width=device-width, initial-scale=1.0": This attribute defines the specific viewport settings.

width=device-width: This setting ensures that the width of the viewport is set to the width of the device's screen. In other words, it makes the webpage responsive to the width of the device.

initial-scale=1.0: This setting sets the initial zoom level when the webpage is first loaded. A value of 1.0 means that the webpage should be displayed at its original size without any initial zooming.

Example: 1.html

```
<!DOCTYPE html>
<html>

<head>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>

<body>

<h2>Setting the Viewport</h2>
<p>This example does not really do anything, other than showing you how to add the viewport meta element.</p>

</body>
</html>
```

Since HTML plays a huge role in defining the structure of a web page, then it definitely has something to do with making websites respond (adapt) to different screen sizes.

You'll need to include the **meta viewport** empty element in the head section of any web page that you want to optimize for different screen sizes.

This tells the browser to render the width of the web page to the exact width of the device. So, if the device has a width of 1200px, the web page width will be 1200px. If the device has a width of 720px, the web page width will be 720px, and so on.

This prevents annoying zooming in and zooming out that mobile phone users experienced in the past, which wasn't good for user experience.

Here is an example of a web page *without* the viewport meta tag, and the same web page *with* the viewport meta tag:

Without the viewport meta tag:



With the viewport meta tag:



B. How to make the Image Responsive?

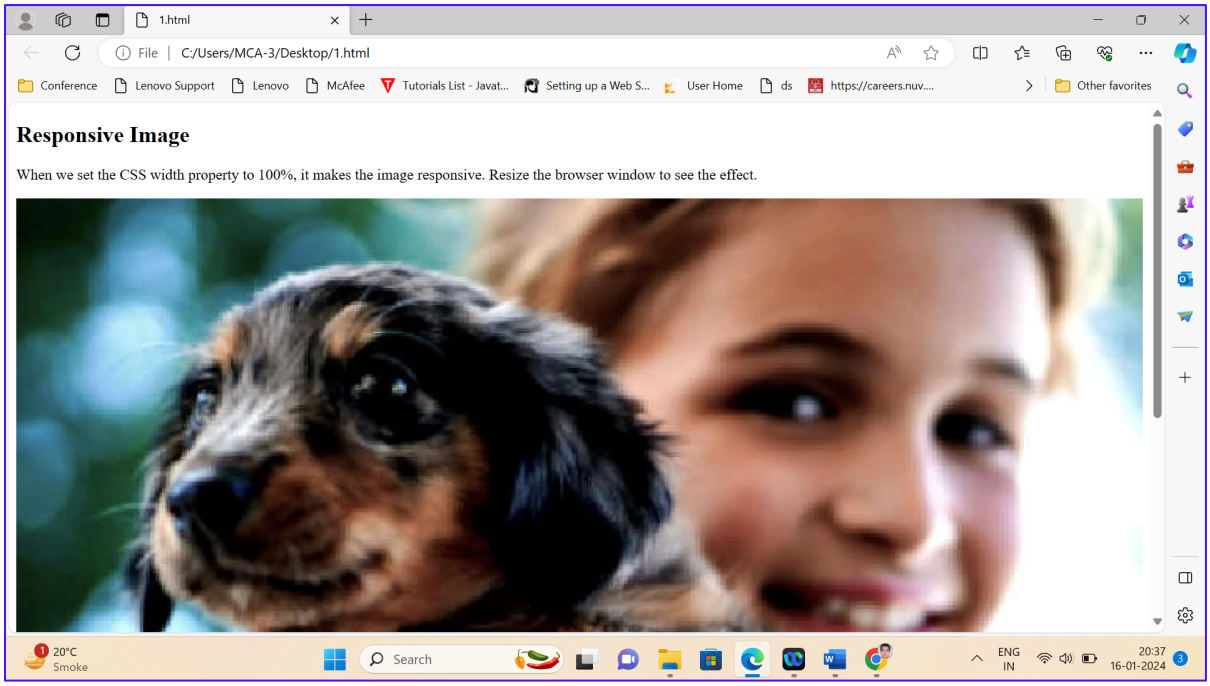
1. By using the width property

Set the CSS width property to 100% to make the image responsive and scale up and down.

Example:

```
<!DOCTYPE html>
<html>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<body>
<h2>Responsive Image</h2>
<p>When we set the CSS width property to 100%, it makes the image responsive. Resize the browser window to see the effect.</p>

<img src="img_girl.jpg" style="width:100%;">(Change the image)
</body>
</html>
```



2. By using the max-width Property

This method is best and most used because it facilitates that the image will scale down if it has to, **but** never scale up to be larger than its original size.

Example:

```
<!DOCTYPE html>
<html>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<body>
<h2>Responsive Image</h2>
<p>"max-width:100%" makes the image responsive and also ensures that the image doesn't get bigger than its original size.</p>
<p>Resize the browser window to see the effect.</p>
<img src="img_girl.jpg" style="max-width:100%;height:auto;"> (Change the image)
</body>
</html>
```

3. Change images according to the browser width

By using the HTML **<picture>** element, you can set two or more images according to the browser width. It will change the picture when you change the browser-size. i.e. desktop and phone.

Example:

```
<!DOCTYPE html>
<html>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<body>
<h2>Change Images Depending on Browser Width</h2>
<p>Resize the browser width and the image will change at 600px and 1500px.</p>
<picture>
<source srcset="img_smallflower.jpg" media="(max-width: 600px)">(Change image)
<source srcset="img_flowers.jpg" media="(max-width: 1500px)">(Change image)
<source srcset="flowers.jpg">
<img src="img_flowers.jpg" alt="Flowers" style="width:auto;">
</picture>
</body>
</html>
```

<picture> element is used to provide multiple sources for an image, allowing the browser to choose the most appropriate one based on the conditions specified.

<source srcset="img_smallflower.jpg" media="(max-width: 600px)">: This source element defines an image source (img_smallflower.jpg) and a media query specifying that it should be used when the browser width is a maximum of 600 pixels.

<source srcset="img_flowers.jpg" media="(max-width: 1500px)">: This source element defines another image source (img_flowers.jpg) and a media query specifying that it should be used when the browser width is a maximum of 1500 pixels.

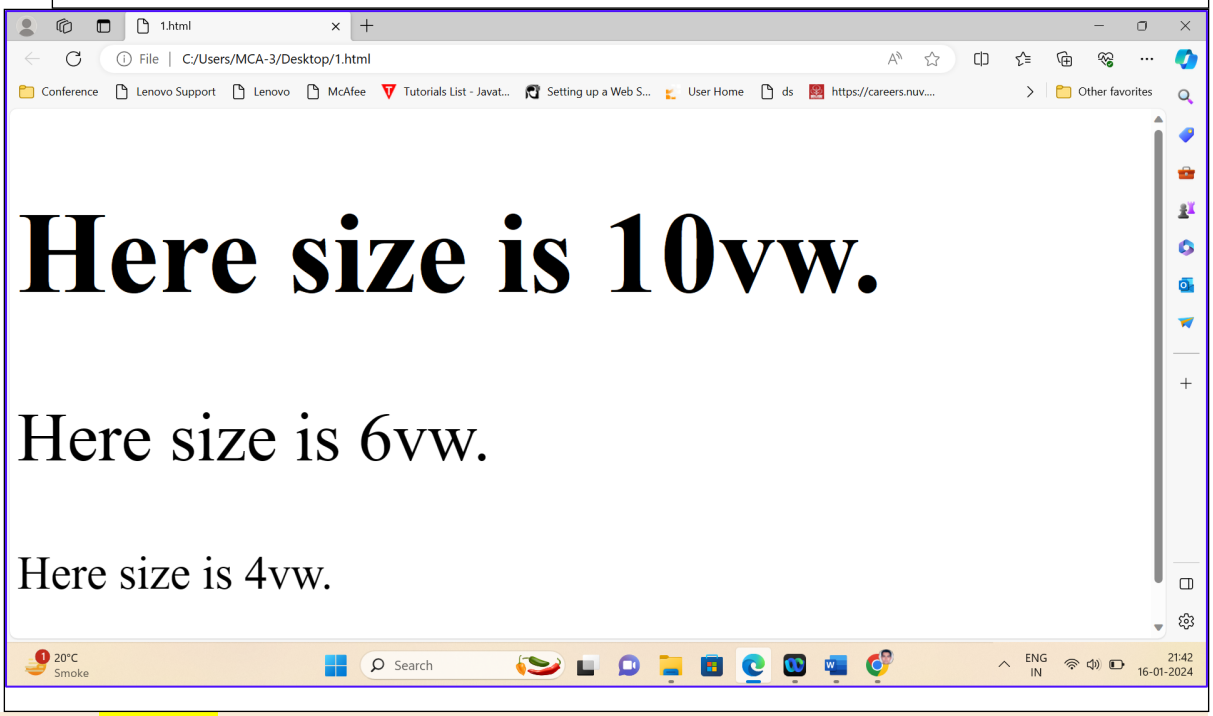
C. Responsive Text-size

We can make the text size responsive by using the "vw" unit. It means viewport-width. By using this, we can make the text size follow the browser window screen.

Example:

```
<!DOCTYPE html>
<html>
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<body>
<h1 style="font-size:10vw;">Here size is 10vw.</h1>
<p style="font-size:6vw;">Here size is 6vw.</p>
<p style="font-size:4vw;">Here size is 4vw.</p>
<p>Resize the browser window to see how the text size changes.</p>
</body>
</html>
```



Note: viewport specifies the browser window size. 1vw = 1% of viewport width. Means, if the viewport is 100cm wide, 1vw is 1.0cm.

Media Queries

Media queries are a feature of CSS (Cascading Style Sheets) that allow you to apply styles to a web page based on the characteristics of the device or the browsing environment.

This **enables** you to create responsive web designs that adapt to different screen sizes, resolutions, and other properties, providing an optimal viewing experience across various devices.

The syntax of a media query typically looks like this:

```
@media media_type and (media_feature) {
  /* CSS rules to be applied when the media query conditions are met */
}
```

@media: This is the keyword that indicates the beginning of a media query.

media_type: Specifies the type of media the query applies to. Common values include screen (for computer screens), print (for printed pages), speech (for screen readers), etc.

media_feature: Specifies the condition or criteria for the styles to be applied. Examples include max-width, min-width, orientation, resolution, and more.

CSS rules: These are the styles that will be applied **if** the conditions specified in the media query are met.

Here's a simple HTML example that incorporates media queries:

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Media Queries Example</title>
  <style>
    body {
      font-family: 'Arial', sans-serif;
      margin: 20px;
```

```
  background-color: #f4f4f4; /* Light gray background color */
}

h1 {
  color: #007bff; /* Blue color */
}

p {
  color: #333; /* Dark gray color */
}

/* Media query for screens larger than 600px */
@media screen and (min-width: 600px) {
  body {
    background-color: #fff; /* White background color for larger screens */
  }

  h1 {
    font-size: 2em; /* Increase font size for larger screens */
  }

  p {
    font-size: 1.2em; /* Increase font size for larger screens */
  }
}

/* Media query for screens larger than 1200px */
@media screen and (min-width: 1200px) {
  h1 {
    font-size: 2.5em; /* Further increase font size for very large screens */
  }

  p {
```

```
    font-size: 1.4em; /* Further increase font size for very large screens */
   }
  }
 </style>
</head>
<body>

 <h1>Responsive Heading</h1>

 <p>
  This is an example paragraph with text that will adapt its style based on the screen size using media queries.
 </p>

</body>
</html>
```

How to add a breakpoint?

In a media query, a breakpoint **refers** to a specific value of a media feature (such as screen width) at which the styles defined within the media query will be applied. **Breakpoints** are commonly used to create responsive designs that adapt to different screen sizes.

The above example of how you can use breakpoints in media queries to apply styles at different screen widths:

In the above example:

The **first media query (min-width: 600px)** represents a breakpoint where styles for screens with a width of 600 pixels or more will be applied.

The **second media query (min-width: 1200px)** represents another breakpoint where styles for screens with a width of 1200 pixels or more will be applied.

These breakpoints define the points at which your layout and styles will change to accommodate different screen sizes. The specific values used for breakpoints will depend on your design and the behavior you want to achieve for different screen sizes.

It's common to define breakpoints based on typical device sizes (e.g., smartphones, tablets, desktops), but the values will ultimately depend on your project's specific needs. Adjust the

breakpoints according to your design requirements and the desired user experience on different devices.

HTML5 Features

1. Audio & Video Tags
2. Header
3. Footer
4. Figure and figcaption
5. Canvas Tag
6. Mark
7. Progress Tag
8. Geolocation API
9. Local Storage
10. Drag and Drop API

HTML5, the latest evolution of HTML, significantly enhances web development and user interactions. It introduces a range of new elements and attributes that reflect typical usage on modern websites. With improved support for multimedia, graphics, and real-time communication, HTML5 paves the way for developing richer, more interactive, and more user-friendly web applications. Its emphasis on semantics helps in creating cleaner, more readable code. Additionally, features like local storage and geolocation support further extend the capabilities of web applications, meeting the growing demands of modern web development. This makes HTML5 a powerful and indispensable tool for today's web developers.

1. Audio & Video Tags

With Audio and Video tags, developers can embed videos or audio into their websites. For styling the video tag, developers can use CSS and CSS3.

Example of the video tag in HTML:

File: 2-Audio_Video_Tags.html

```
<!DOCTYPE html>
<html lang="en">

<head>
<title>HTML5</title>
</head>
<body>
<!-- Code to setup video -->

<video width = "300" height = "200" controls autoplay>
```

```
<source src = "./dog.mp4" type ="video/mp4" />
</video>

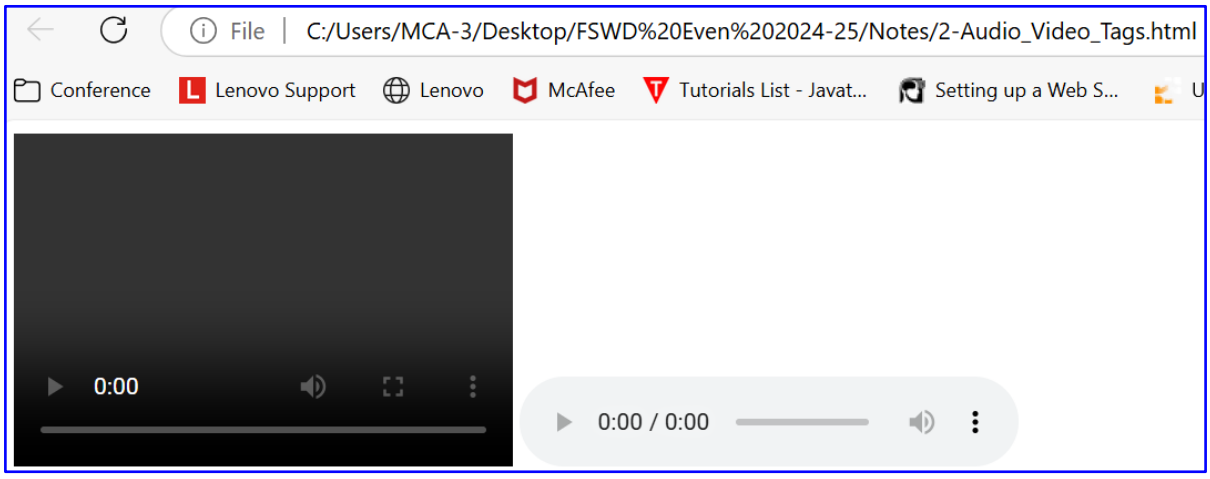
<!-- Code to setup audio -->

<audio controls>
<source src="dog.mp3" type="audio/mp3">
</audio>
</body>
</html>
```

In the above HTML code:

- width (300 pixels)and height (200 pixels) are for setting the dimensions for the video element
- controls attribute creates buttons for playback (Play, Pause, etc.)
- src (source) tag provides the URL or media source (`./` indicates that the file `dog.mp4` is located in the current directory relative to the HTML file.)
- type represents the video type

output:



2. Header

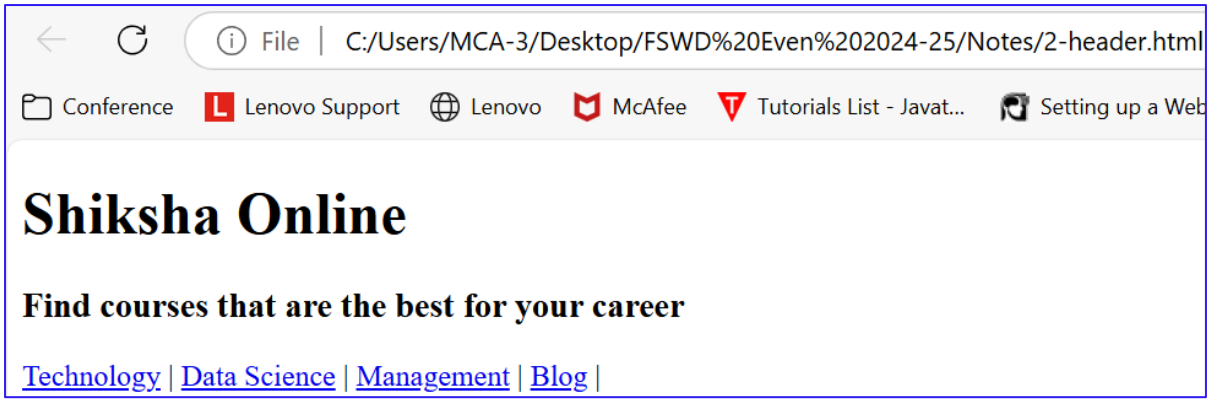
The element in HTML5 contains introductory content. It can include a set of introductory or navigational support on a website. Typically, the header element contains heading elements, logos or icons, a search form, author's information, etc.

Example of header element in HTML5: 2-header.html

```
<!DOCTYPE html>
<html>
```

```
<body>
  <h1>Shiksha Online</h1>
  <h3> Find courses that are the best for your career </h3>
    <header>
    <a href=
"https://www.shiksha.com/online-courses/technology-courses-certification-training-ct139">
    Technology</a> |
    <a href=
"https://www.shiksha.com/online-courses/data-science-courses-certification-training-ct123">
    Data Science</a> |
    <a href=
"https://www.shiksha.com/online-courses/management-courses-certification-training-ct135">
    Management</a> |
    <a href=
"https://www.shiksha.com/online-courses/articles/">
    Blog</a> |
  </header>
</body>
</html>
```

Output:



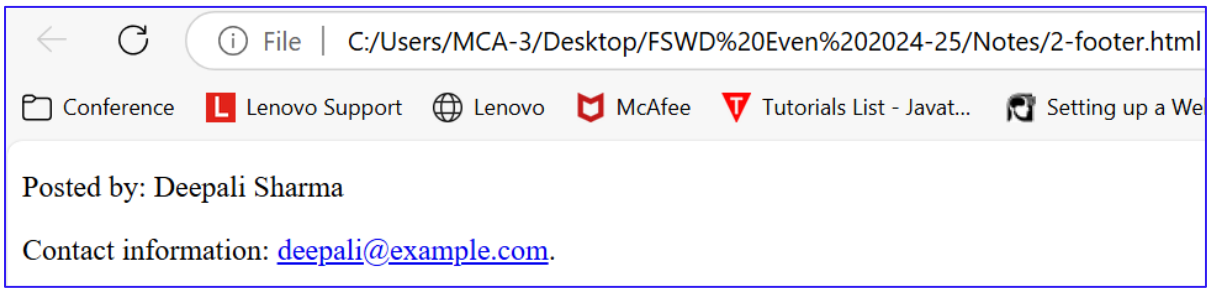
3. Footer

The tag defines the footer of a document or a section. Typically, the footer contains information related to the author, copyright, contact information, sitemap, related documents, back-to-top links, etc. There can be more than one footer element in one document.

Example of the footer in HTML5: 2-footer.html

```
<!DOCTYPE html>
<html>
  <body>
      <footer>
      <p>Posted by: Deepali Sharma</p>
      <p>Contact information: <a href="mailto:deepali@example.com">
      deepali@example.com</a>.</p>
    </footer>
  </body>
</html>
```

Output:



4. Figure and figcaption

These elements help users to insert an image with its caption. The **figcaption** tag is used to add a description for the image. It is typically used within a <figure> element to associate the caption with the image.

Example of figcaption: 2-figcaption.html

```
<!DOCTYPE html>
<html>
<body>
<figure>
  <img src="red tulips.jpg"
    alt="Red Tulips">
  <figcaption>Red Tulips in a Garden</figcaption>
</figure>
</body>
</html>
```

Output:



5. Canvas Tag

The canvas tag in HTML5 helps users draw graphics or images on the fly using JavaScript. We can use it for drawing paths, boxes, circles, adding images, etc. The canvas tag has two attributes: width and height for setting the width and height of the canvas.

Example of canvas tag in HTML5:

```
<!DOCTYPE html>
<html>
  <body>
    <canvas id="Canvas1" width="400" height="100" style="border:2px solid;">
    </canvas>
  </body>
</html>
```

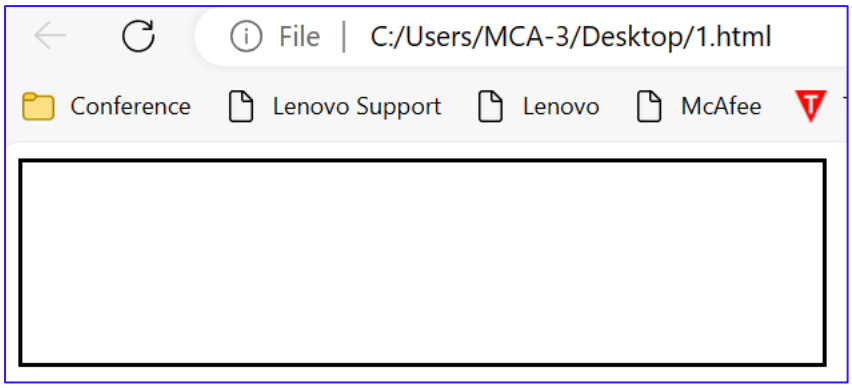
width="400":

- Sets the width of the canvas in pixels.
- Default width (if not specified) is 300px.

height="100":

- Sets the height of the canvas in pixels.
- Default height (if not specified) is 150px.

Output:



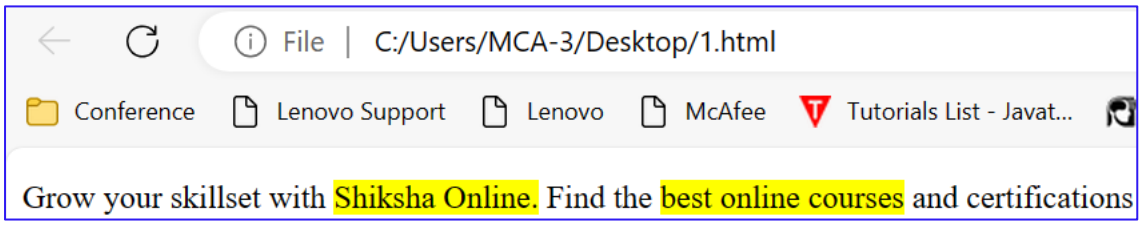
6. Mark

The <mark> element highlights a particular text of special interest to the user in an HTML document.

Example of the mark element:

```
<!DOCTYPE html>
<html>
    <body>
      <p> Grow your skillset with <mark>Shiksha Online.</mark> Find the
<mark>best online courses</mark> and certifications in management, technology,
programming, and data science.</p>
    </body>
</html>
```

Output:



7. Progress Tag

The progress tag helps users check the progress of a task during the execution. Users will need to define the dynamically changing values of the progress bar with the scripts (JavaScript).

Example of progress tag in HTML5: 2-progress.html

```
<!DOCTYPE html>
<html>
 <body>
  <span>Loading:</span>
  <progress value="25" max="100"></progress>
 </body>
```

```
</html>
```

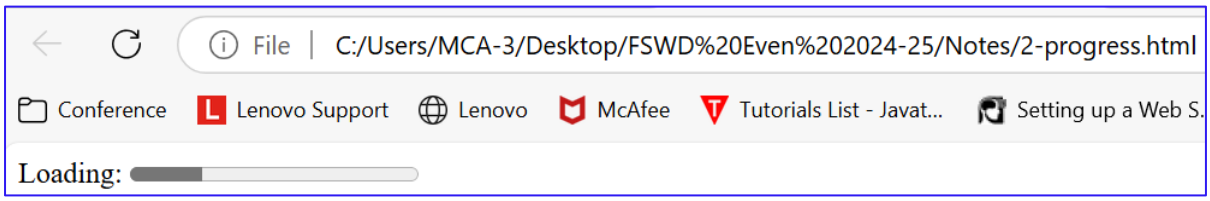
Attributes:

- value="25":
  - Represents the current progress of the task (e.g., 25% completed).
- max="100":
  - Defines the maximum value the progress can reach (e.g., 100%).

Behavior:

- The progress bar visually fills to 25% of its total width because the value is set to 25 and the max is set to 100.

Output:



8. Geolocation API

The Geolocation API is a standard web API that allows web applications to request the user's location information.
Modern web browsers support the Geolocation API, allowing websites to request and use the user's location data.

JavaScript functions, such as navigator.geolocation.getCurrentPosition, facilitate this process.

Below example will display the latitude and longitude of the user's location on a web page.

The Geolocation API in web browsers typically **relies** on location information obtained through a combination of methods, including GPS, Wi-Fi positioning, and cell tower triangulation.

File : 2-Geolocation.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Geolocation Example</title>
  <style>
    #location-info {
      font-size: 18px;
      margin-top: 20px;
    }
  </style>
</head>
<body>

<h1>Geolocation Example</h1>

<div id="location-info">
  <strong>Latitude:</strong> <span id="latitude"></span><br>
  <strong>Longitude:</strong> <span id="longitude"></span>
</div>

<script>
  // Check if the browser supports the Geolocation API
  if (navigator.geolocation) {
    // Request the user's current location
    navigator.geolocation.getCurrentPosition(showPosition, showError);
  } else {
    // Display an error message if the Geolocation API is not supported
    showError("Geolocation is not supported by this browser.");
  }

  // Callback function to handle the successful retrieval of the user's location
  function showPosition(position) {
    // Extract latitude and longitude from the position object

    var latitude = position.coords.latitude;
    var longitude = position.coords.longitude;

    // Display the latitude and longitude on the web page
    document.getElementById("latitude").textContent = latitude.toFixed(6);
    document.getElementById("longitude").textContent = longitude.toFixed(6);
  }
```

```
  // Callback function to handle errors during the retrieval of the user's location
  function showError(error) {
    var errorMessage;

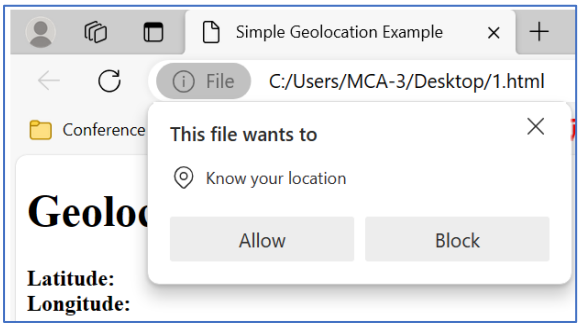
    switch (error.code) {
      case error.PERMISSION_DENIED:
        errorMessage = "User denied the request for Geolocation.";
        break;
      case error.POSITION_UNAVAILABLE:
        errorMessage = "Location information is unavailable.";
        break;
      case error.TIMEOUT:
        errorMessage = "The request to get user location timed out.";
        break;
      case error.UNKNOWN_ERROR:
        errorMessage = "An unknown error occurred.";
        break;
      default:
        errorMessage = "An unspecified error occurred.";
        break;
    }

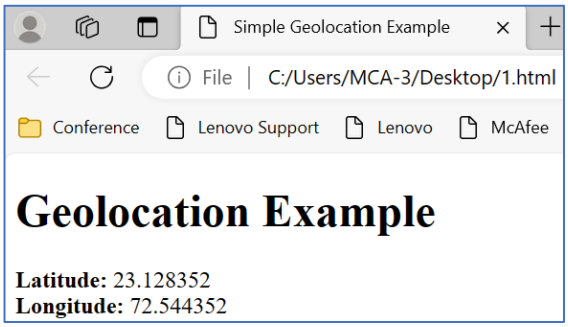
    // Display the error message on the web page
    alert(errorMessage);
  }
</script>

</body>
</html>
```

- The script checks if the browser supports Geolocation using **navigator.geolocation**.
- If supported, it calls **navigator.geolocation.getCurrentPosition** to request the user's current location, passing two callback functions: showPosition for successful retrieval and showError for handling errors.
- If Geolocation is not supported, an alert is displayed notifying the user.
- The showPosition function extracts the latitude and longitude from the position object and updates the corresponding spans in the HTML to display these values.

- The showError function handles different types of errors that might occur during the Geolocation process and displays an alert with an appropriate message.





9. Local Storage

It is a modern feature of HTML and several browsers that typically store data in the user's browsers and can access them with the help of JavaScript APIs. This feature is useful for creating offline applications where data is need to be stored locally.

Moreover, using this feature you can reduce the transactions between the application and the backend server, creating a fast application. However, there are some limitations, such as there is limited storage, and you can not access more storage than that.

Local Storage is a simple **key-value** storage mechanism provided by web browsers to store data persistently on a user's device. Here's a basic example using JavaScript to interact with the Local Storage:

File : 2-local-starage.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Local Storage Example</title>
</head>
<body>

<h1>Local Storage Example</h1>

<button onclick="saveData()">Save Data</button>
<button onclick="retrieveData()">Retrieve Data</button>

<script>
  // Function to save data to Local Storage
  function saveData() {
    // Get data from user (for simplicity, we're using a prompt here)
```

```
    var userInput = prompt("Enter some data:");

    // Check if the user entered something
    if (userInput) {
      // Save data to Local Storage with a specific key
      localStorage.setItem("userData", userInput);
      alert("Data saved to Local Storage!");
    } else {
      alert("Please enter some data.");
    }
  }

  // Function to retrieve data from Local Storage
  function retrieveData() {
    // Retrieve data from Local Storage using the key
    var storedData = localStorage.getItem("userData");

    // Check if there is any stored data
    if (storedData) {
      alert("Data retrieved from Local Storage: " + storedData);
    } else {
      alert("No data found in Local Storage.");
    }
  }
</script>

</body>
</html>
```

saveData Function:

This function is executed when the "Save Data" button is clicked.

It uses the prompt function to get input from the user (data to be saved).

If the user enters data, it uses **localStorage.setItem** to save the data with the key "**userData**" to Local Storage.

An alert is displayed indicating that the data has been saved, or an alert prompts the user to enter data if nothing is entered.

localStorage.setItem("userData", userInput);: This line of code is using the **localStorage.setItem** method to store data in the Local Storage of the browser.

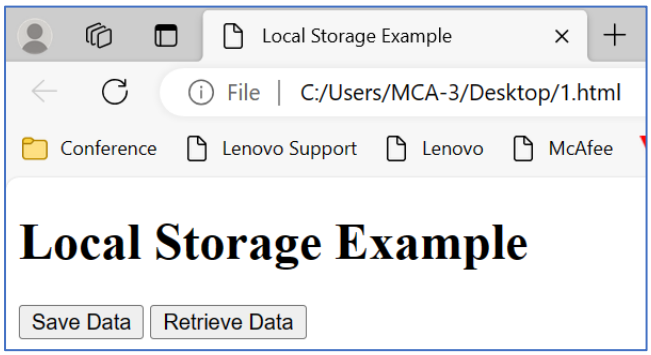
- **"userData"**: This is the **key** under which the data will be stored. The key is a string that acts as a reference to retrieve the data later. In this case, the key is set to "userData".
- **userInput**: This is the **data** that you want to store. It could be a string, number, object, or any other data type that can be represented as a string. The data provided by the user (presumably stored in a variable called **userInput**) is being stored in the Local Storage.

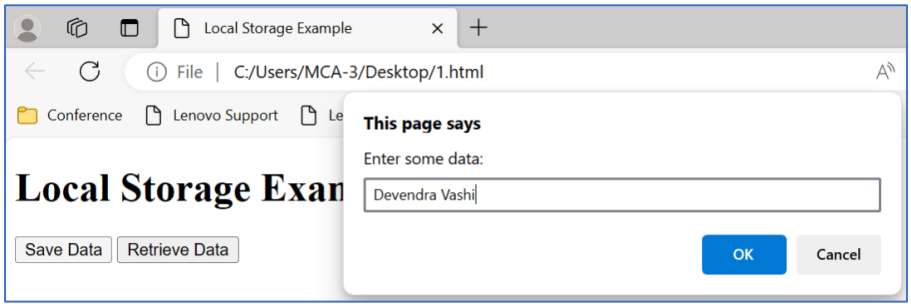
retrieveData Function:

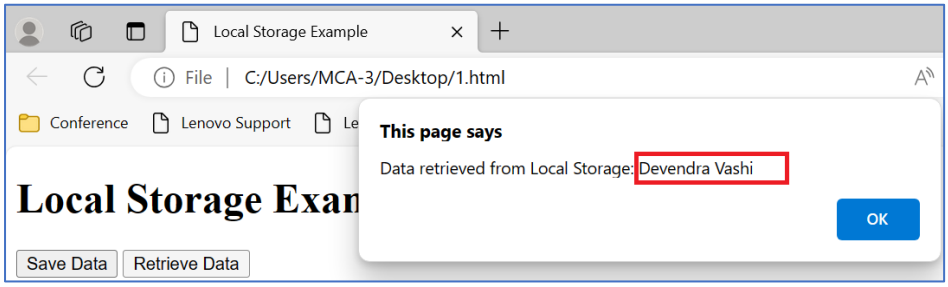
This function is executed when the "Retrieve Data" button is clicked.

It uses **localStorage.getItem** to retrieve the previously saved data from Local Storage using the key "**userData**."

If data is found, an alert is displayed with the retrieved data. If no data is found, an alert indicates that no data was found.





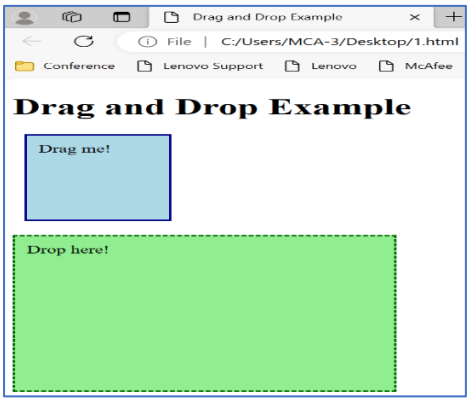


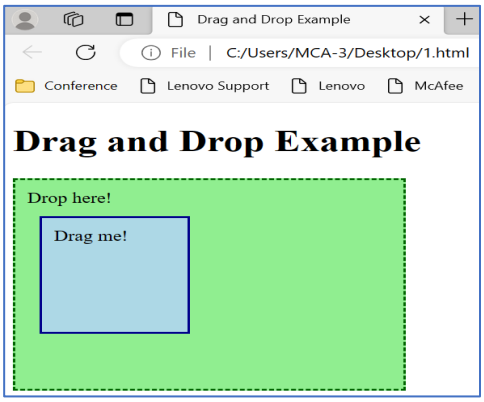
10. Drag and Drop API

Drag and drop is among the most unique features of HTML5 that allow you to **grab** any element in the Document Object Model (DOM) and **drop** it to a different location.

To create an element able to drag and drop, **set** the attribute "**draggable**" on the tag and put its value to **true**.

The Drag and Drop API in HTML allows you to make elements on a web page draggable and define drop zones for them. Here's a simple example demonstrating the Drag and Drop API:





File: 2-drag-drop.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Drag and Drop Example</title>
  <style>
    #dragElement {
      width: 100px;
      height: 100px;
      background-color: lightblue;
      border: 2px solid darkblue;
      margin: 10px;
      padding: 10px;
      cursor: move;
    }
    #dropZone {
      width: 300px;
      height: 200px;
```

```
        background-color: lightgreen;
        border: 2px dashed darkgreen;
        margin-top: 20px;
        padding: 10px;
      }
    </style>
</head>
<body>

<h1>Drag and Drop Example</h1>

<div id="dragElement" draggable="true" ondragstart="dragStart(event)">
  Drag me!
</div>

<div id="dropZone" ondragover="allowDrop(event)" ondrop="drop(event)">
  Drop here!
</div>

<script>
  // Function to handle the drag start event
  function dragStart(event) {
    // Set data to be transferred during drag
    event.dataTransfer.setData("text/plain", event.target.id);
  }

  // Function to allow dropping on the drop zone
  function allowDrop(event) {
    event.preventDefault(); //By default, when a form is submitted, the page reloads. Using
                                    event.preventDefault()
  }

  // Function to handle the drop event
  function drop(event) {
    event.preventDefault();

    // Get the dragged element's id from the data transfer
    var draggedElementId = event.dataTransfer.getData("text/plain");
```

```
    // Get the dragged element by id
    var draggedElement = document.getElementById(draggedElementId);

    // Append the dragged element to the drop zone
    document.getElementById("dropZone").appendChild(draggedElement);
  }
</script>

</body>
</html>
```

When you run this code, you'll see a draggable element ("Drag me!") that can be dragged and dropped into the specified drop zone ("Drop here!"). The JavaScript functions handle the drag-and-drop events, allowing for interactive behavior on the webpage.

- The HTML document includes the necessary meta tags and a title.
- Inside the **<style>** tag, there are CSS styles for the draggable element (**#dragElement**) and the drop zone (**#dropZone**).

- The "dragElement" div is created with an id, and the **draggable** attribute is set to "true" to make it draggable.
- The **ondragstart** attribute is set to the function **dragStart(event)**, which will be called when the drag starts.

- The "dropZone" div is created with an id.
- The **ondragover** attribute is set to the function **allowDrop(event)**, which allows dropping by preventing the default behavior.
- The **ondrop** attribute is set to the function **drop(event)**, which handles the drop event.

- **dragStart Function:**
  - This function is called when the drag starts (**ondragstart**).
  - It sets data to be transferred during the drag using **event.dataTransfer.setData**. In this case, it sets the id of the dragged element.
- **allowDrop Function:**
  - This function is called during the **ondragover** event.
  - It prevents the default behavior (allowing dropping) using **event.preventDefault()**.

- **drop Function:**
  - This function is called during the **ondrop** event.
  - It prevents the default behavior using **event.preventDefault()**.
  - It retrieves the dragged element's id from the data transfer using **event.dataTransfer.getData**.
  - It gets the dragged element by id using **document.getElementById**.
  - It appends the dragged element to the drop zone (**#dropZone**) using **appendChild**.

Bootstrap :

What is Bootstrap?

Bootstrap is an open-source **front-end framework** developed by **Twitter**. It is a **collection of HTML, CSS, and JavaScript** tools and components that helps developers build responsive and mobile-first web applications.
The framework simplifies the process of designing and styling web pages by providing a set of pre-built components and styles that can be easily customized and adapted.

Key features of Bootstrap include:

I. **Responsive Design:** Bootstrap is designed with a mobile-first approach, ensuring that websites built using the framework are responsive and adapt well to various screen sizes, from small mobile devices to large desktops.

II. **Grid System:** Bootstrap includes a responsive grid system that allows developers to create flexible and dynamic layouts. The grid system is based on a **12**-column layout, making it easy to create responsive designs and organize content.

III. **CSS Components:** Bootstrap provides a variety of CSS components, such as **n**avigation bars, **b**uttons, **f**orms, **t**ypography, and more.
These components are styled and pre-designed, allowing developers to easily integrate them into their projects.

IV. **JavaScript Plugins:** Bootstrap includes a set of JavaScript plugins that enhance the functionality of web pages. These plugins cover common features like modals, carousels, tooltips, and popovers, among others.

V. **Customization:** While Bootstrap provides a default set of styles and components, it is highly customizable. Developers can use the Bootstrap source files and customize the framework according to their project's specific needs. Bootstrap can also be integrated with other CSS frameworks or pre-processors like Sass or Less (are preprocessor scripting languages that extend the capabilities of CSS).

VI. **Community and Documentation:** Bootstrap has a large and active community, which means there is a wealth of resources, tutorials, and third-party themes available. The official documentation is comprehensive and provides detailed information on how to use each component, making it easier for developers to get started.

VII. **Cross-Browser Compatibility:** Bootstrap is designed to be compatible with various web browsers, ensuring consistent and reliable performance across different platforms.

What is new in Bootstrap 5?

I. **Major Rewrite of the Entire Framework:** Bootstrap 5 represents a major rewrite of the entire framework. This involves significant changes to the underlying codebase, structure, and architecture. The rewrite is likely aimed at improving performance, modularity, and overall code quality.

II. **Brand New Look and Feel:** Bootstrap 5 introduces a fresh and updated visual design, providing a modern and aesthetically pleasing look. This includes improvements to default styling, layout components, and other design elements, giving websites a contemporary appearance.

III. **Brand New Logo:** Along with the major rewrite, Bootstrap 5 revealed a new logo. A new logo often signifies a rebranding effort and serves as a visual representation of the framework's evolution and identity.

IV. **New Typography:** Bootstrap 5 includes updates to typography, focusing on enhancing the readability and visual appeal of text across different devices. This may involve changes to font styles, sizes, and spacing for improved legibility.

V. **No jQuery is Required Anymore:** One notable change in Bootstrap 5 is the removal of the jQuery dependency. Bootstrap now relies on inherent JavaScript for its functionality, making it more lightweight and aligning with modern web development practices.

VI. **Dropping Support for Internet Explorer (v5):** Bootstrap 5 has discontinued support for Internet Explorer 11. By dropping support for this older browser, Bootstrap can take advantage of modern web standards and technologies, allowing for more efficient development and better performance.

VII. **New Theme Colors:** Bootstrap 5 introduces a new set of theme colors, offering developers a refreshed color palette to use in their projects. These colors can be applied to various components, contributing to a more diverse and modern visual style.

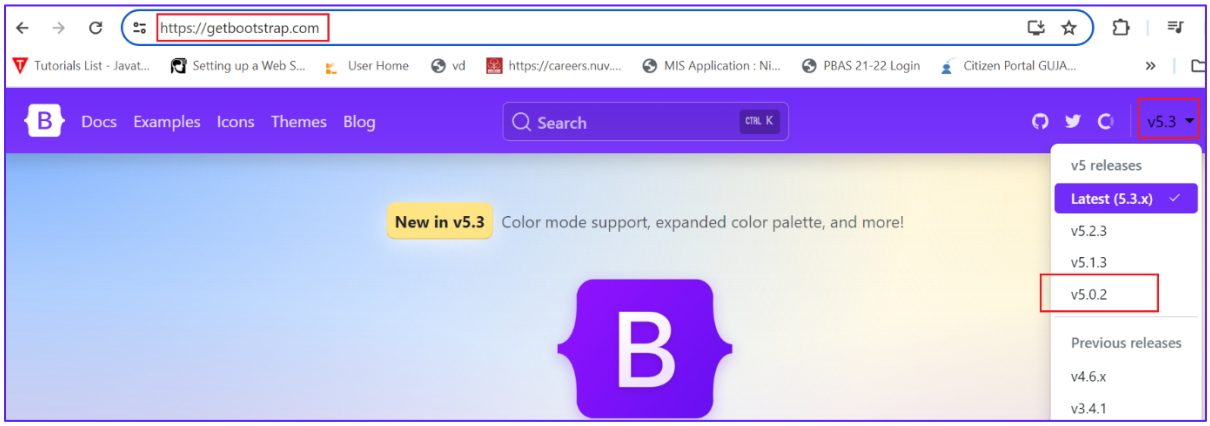
VIII. **Enhanced Grid:** The grid system is a fundamental aspect of Bootstrap, and in version 5, it likely received enhancements for improved flexibility and control over layout structures. This could involve new utility classes and features to make responsive design even more powerful.

IX. **Offcanvas:** The offcanvas feature in Bootstrap 5 provides a way to create side menus or panels that are hidden off-screen and can be revealed on demand. This enhances the navigation options available in Bootstrap, improving user experience and design possibilities.

X. **Bootstrap 5 has its Own Icon System:** Bootstrap 5 comes with its own set of icons, providing a built-in icon system for developers to use in their projects. This eliminates the need to rely on external icon libraries and allows for easier customization and integration.

Bootstrap has gained widespread adoption in the web development community due to its ease of use, flexibility, and ability to speed up the development process. Whether you are a beginner or an experienced developer, Bootstrap can be a valuable tool for creating modern and responsive web applications.

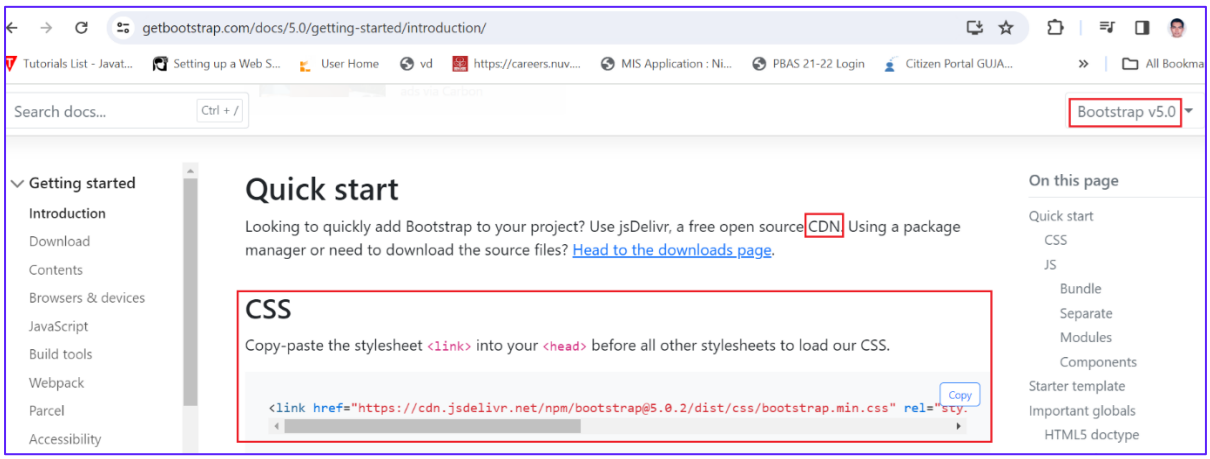
Bootstrap 5 Installation:



A. Use of CDN link for bootstrap: (How to use bootstrap online?)

Using a Content Delivery Network (CDN) link for Bootstrap is a convenient way to include the Bootstrap framework in your web project.

Bootstrap is a popular front-end framework that simplifies the process of designing responsive and visually appealing web pages. Here's how you can use a CDN link for Bootstrap:



Include the CDN Link in Your HTML File:

Add the CDN Link tag in the <head> section of your HTML file to include the Bootstrap CSS file

Make sure to replace **5.0.2** with the version you want to use. You can find the latest version on the Bootstrap website or the chosen CDN provider's documentation.

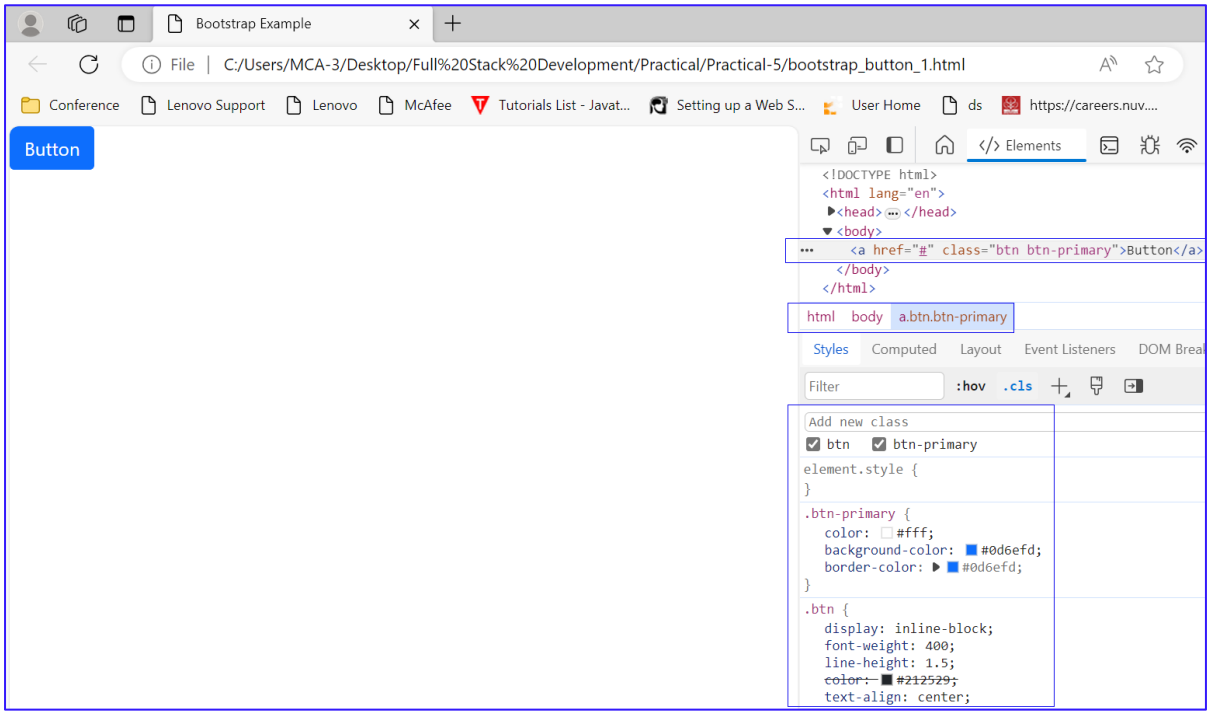
Example: 2-bootstrap_button_1_CDN.html :Creating a Button using CDN (Internet service is required)

```
<!DOCTYPE html>
<!DOCTYPE html>
<html lang="en">
<head>
 <title>Bootstrap Example</title>
 <meta charset="utf-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/
bootstrap.min.css" rel="stylesheet" integrity="sha384-EVSTQN3/
azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
crossorigin="anonymous">

</head>
<body>
<a href="#" class="btn btn-primary">Button</a>
</body>
</html>
```

Output:



Advantages of using a CDN link:

By using a CDN link, you benefit from faster loading times because users can potentially retrieve the Bootstrap files from a server geographically closer to them. Additionally, it reduces the burden on your web server.

By using CDN links, you ensure that your web page is always using the latest version of Bootstrap without having to download and host the files yourself. Keep in mind that using a CDN requires an internet connection for your users to load the Bootstrap files.

B. Use of compiled CSS file for bootstrap: (How to use bootstrap offline?)

To use Bootstrap offline, you need to download the Bootstrap files and host them locally in your project.

Here's a step-by-step guide on how to use Bootstrap offline:

I. Download Bootstrap Files:

Visit the official Bootstrap website (https://getbootstrap.com/) and go to the "Download" section. There, you can choose to download the compiled CSS and JavaScript files. Download the ZIP file and extract its contents to a location in your project.

II. Organize Bootstrap Files:

Once you've extracted the contents of the ZIP file, you'll find folders such as css and js. Organize these folders in a way that makes sense for your project structure.

III. Include Bootstrap CSS in Your HTML File:

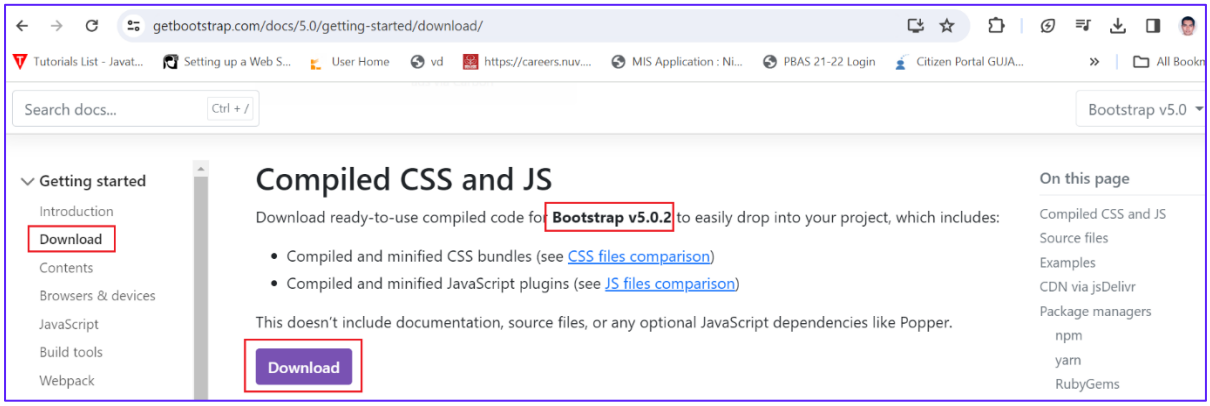
Add a link to the Bootstrap CSS file in the <head> section of your HTML file. Adjust the path based on where you've placed the CSS file within your project:

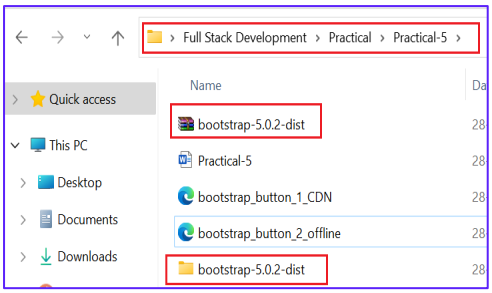
IV. Ensure Proper File Paths:

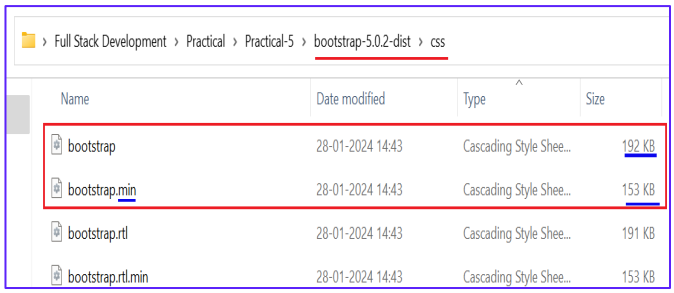
Make sure the file paths in your HTML file accurately reflect the location of the Bootstrap files within your project.

V. Test Your Project:

Open your HTML file in a web browser to ensure that Bootstrap is applied correctly. If everything is set up correctly, you should see the Bootstrap styles being applied to your elements.

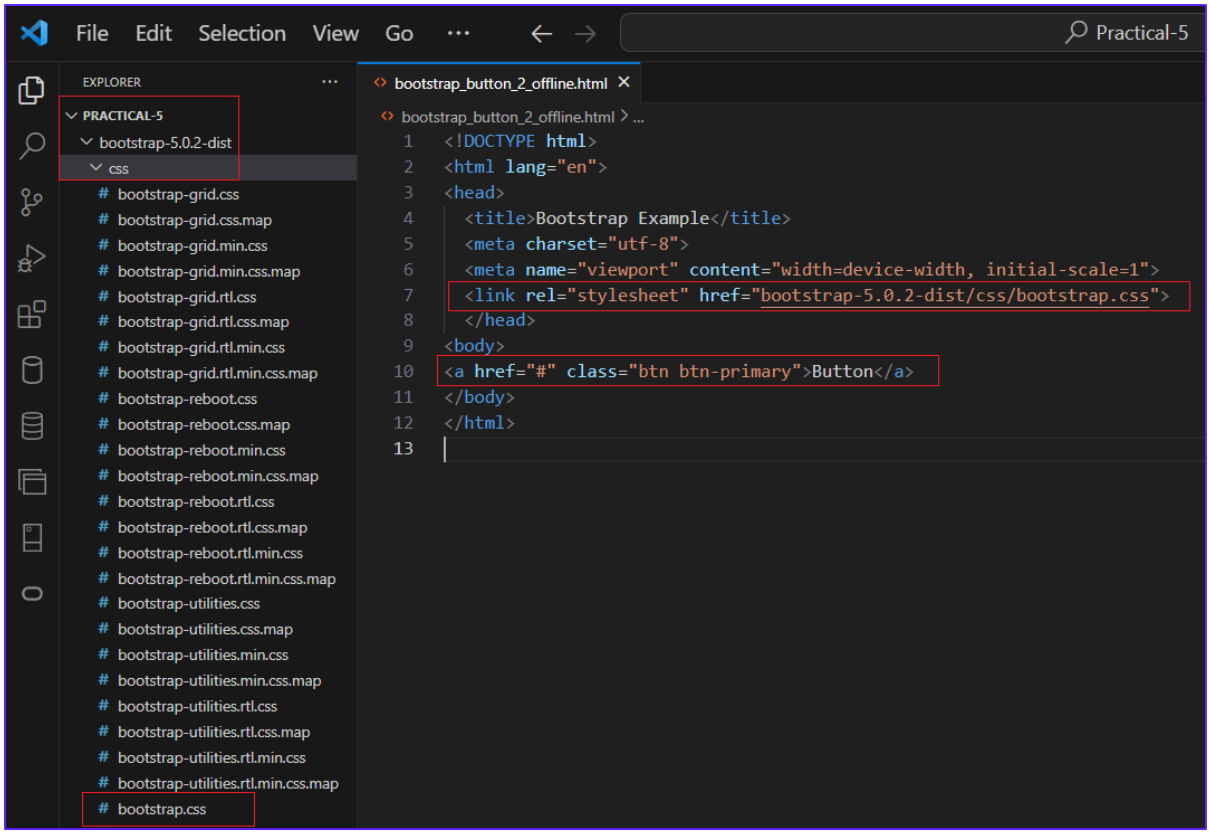




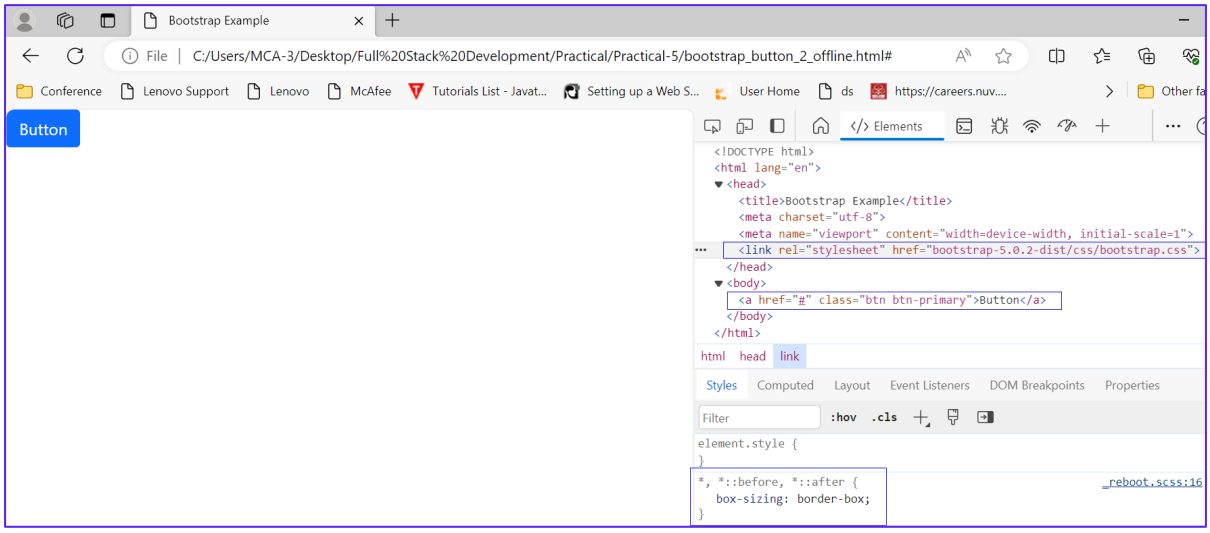


Example: Creating a Button using offline bootstrap:

```
<!DOCTYPE html>
<html lang="en">
<head>
 <title>Bootstrap Example</title>
 <meta charset="utf-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <link rel="stylesheet" href="bootstrap-5.0.2-dist/css/bootstrap.css">
 </head>
<body>
<a href="#" class="btn btn-primary">Button</a>
</body>
</html>
```



Output:



Advantages of using compiled CSS files offline for Bootstrap:

Using the compiled CSS file offline for Bootstrap offers several advantages, including:

1. Offline Availability:

When you use the compiled CSS file offline, your web application can still function even without an internet connection. This is crucial for scenarios where users may not always

have access to the internet, such as in offline applications or on local development environments.

2. **Reduced Dependency on External Servers:**
By hosting the Bootstrap CSS file locally, you reduce dependency on external servers or Content Delivery Networks (CDNs). This can lead to faster loading times, as the CSS file is fetched directly from your server, eliminating potential delays caused by external server issues or network latency.

3. **Improved Performance:**
Local hosting of the compiled CSS file can result in improved performance since the file is retrieved from the local file system. This reduces the need for network requests, leading to faster rendering of styles and improved user experience.

4. **Customization and Theming:**
Storing and using the compiled CSS file locally allows you to customize Bootstrap's styles to match your project's specific requirements. You can modify the CSS file directly or use Bootstrap's customization features to tailor the framework to your design needs.

5. **Version Control:**
Hosting the compiled CSS file locally facilitates version control. You have control over which version of Bootstrap you are using, and updates can be managed intentionally. This ensures consistency in styling across your application, and you won't inadvertently be affected by updates to external CDNs.

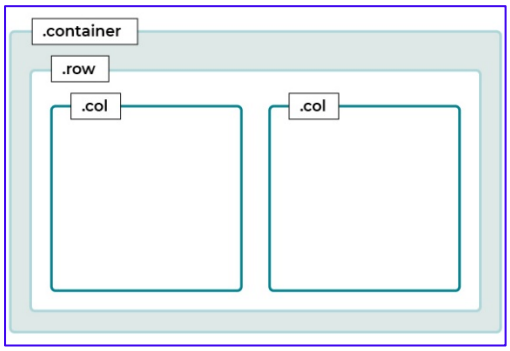
6. **Security and Privacy:**
Storing and using Bootstrap files locally can enhance the security and privacy of your application. It reduces the risk of external entities tampering with or intercepting the CSS file during transmission, providing a more secure environment for your web application.

7. **Isolation from CDN Outages:**
If your web application relies on external CDNs for assets, it might be vulnerable to outages or disruptions in those CDNs. Hosting Bootstrap files offline ensures that your application's styling remains intact even if the CDN experiences issues.

8. **Development and Testing:**
Local hosting is advantageous during the development and testing phases. It allows you to work on your project without requiring an internet connection, simplifying the development workflow, especially in isolated or restricted environments.

While there are advantages to using the compiled CSS file offline, it's essential to consider trade-offs, such as the manual effort required to update Bootstrap versions and potential increases in file size when compared to using a CDN. Ultimately, the choice between local hosting and using a CDN depends on your project's specific requirements and constraints.

Bootstrap 5 Containers:



Bootstrap 5 **Containers** are a core Bootstrap building piece that contains and align your content within a specific device or viewport.

The bootstrap 5 **container** contains responsive width with the required **row and column** for the advanced features and content.

The container class works as fixed and responsive on the web page.

Bootstrap 5 to Create Containers

Containers are the most fundamental Bootstrap layout elements necessary when using the grid system.

Containers are mostly used to enclose material with padding.

In the case of a fixed-width layout, they are also used to center the content horizontally on the page.

Bootstrap 5 has three types of containers:

A. **Container**: The ".container" class has a **maximum** width **at each responsive breakpoint**.

B. **Container-fluid**: The ".container-fluid" class has a width of 100 percent **at all** breakpoints.

C. **Responsive Container**: The ".container-breakpoint" has a width of 100 percent **until** the specified breakpoint.

A. Bootstrap 5 "container" class:

The container class contains content, navbar, image slider, and other functions with a grid system. It is a **fixed-width** container for web pages. The container class provides spaces inside the body section. It's **placed in the center of the body** section of the html page.

Example

The following example shows container in body tag with content. Bootstrap 5 uses the "container" class with "bg-info" background color. We can add row, columns and other features in the container tag.

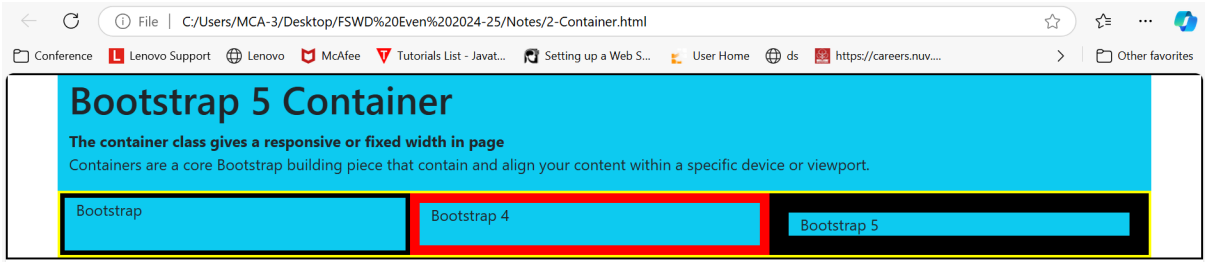
File: 2-Container.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<title> Bootstrap 5 container Example </title>
<meta charset = "utf-8">
<meta name = "viewport" content = "width=device-width, initial-scale=1">
    <link href = "https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel = "stylesheet">
    <script src = "https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js">
</script>
<style>
body{
border:2px solid black;
}
</style>
</head>
<body>
<div class="container bg-info">
    <h1> Bootstrap 5 Container </h1>
    <b> The container class gives a responsive or fixed width in page </b>
    <p> Containers are a core Bootstrap building piece that contain and align your content within a specific device or viewport. </p>
```

```
<div class = "row" style="border : 3px solid Yellow;">
<div class = "col" style="border : 5px solid black;"> Bootstrap </div>
<div class = "col" style="border : 10px solid red;"> Bootstrap 4 </div>
<div class = "col" style="border : 20px solid black;"> Bootstrap 5 </div>
</div>
</div>
</body>
</html>
```

Output:

The following image shows the output of the container class example.



B. Bootstrap 5 "container-fluid" class

The container-fluid class contains content, navbar, image slider, and other functions with a grid system. The container class acquires space inside of the body section. If we need to use all space of the web page, then use the container-fluid class. It is a full-width container for web pages.

Example

The following example shows container in body tag with content. Bootstrap 5 uses the "container-fluid" class with "bg-info" background color. We can add fixed or responsive row, columns and other features in the container-fluid element.

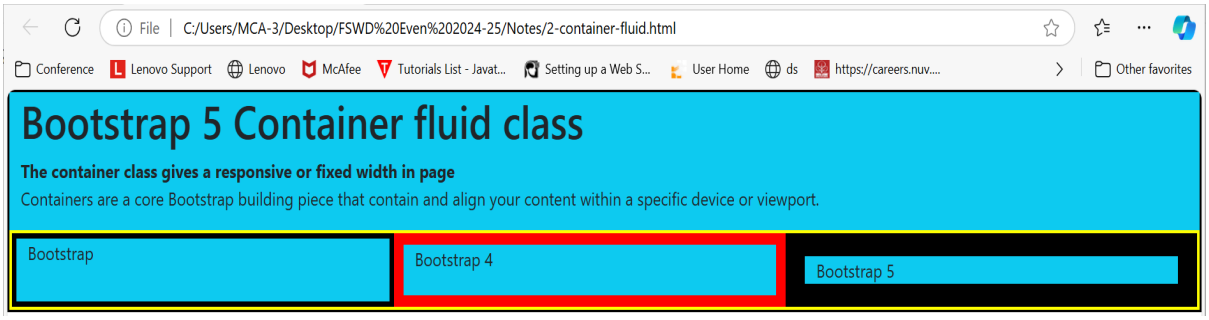
File: 2-container-fluid.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<title> Bootstrap 5 container fluid Example </title>
<meta charset = "utf-8">
<meta name = "viewport" content = "width=device-width, initial-scale=1">
```

```
<link href = "https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel = "stylesheet">
<script src = "https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js">
</script>
<style>
body{
border:2px solid black;
}
</style>
</head>
<body>
<div class="container-fluid bg-info">
<h1> Bootstrap 5 Container fluid class </h1>
<b> The container class gives a responsive or fixed width in page </b>
<p> Containers are a core Bootstrap building piece that contain and align your conte
nt within a specific device or viewport. </p>
<div class = "row" style="border : 1px solid Yellow;">
<div class = "col" style="border : 5px solid black;"> Bootstrap </div>
<div class = "col" style="border : 10px solid red;"> Bootstrap 4 </div>
<div class = "col" style="border : 20px solid black;"> Bootstrap 5 </div>
</div>
</div>
</body>
</html>
```

Output:

The following image shows the output of the container class example.



C. Bootstrap 5 responsive container class

OR container-breakpoint class

The container-breakpoint class contains content and other functions with the grid system. The **breakpoint** places the size of the **fixed-width for particular devices**. The container class acquires space inside of the body section. The responsive container shows the container as per device width.

The following table shows breakpoint with width size.

Classes container-breakpoint	**Extra Small < 576px (mobile)**	**Small (sm) ≥ 576px (mobile**	**Medium (md) ≥ 768px (Tablet)**	**Large(lg) ≥ 992px (Laptop**	**X-Large (xl) ≥ 1200px**	**XX-Large (xxl) ≥ 1400px**
"container" class	100%	540px	720px	960px	1140px	1320px
"container-**sm**"	100%	540px	720px	960px	1140px	1320px
"container-**md**"	100%	100%	720px	960px	1140px	1320px
"container-**lg**" class	100%	100%	100%	960px	1140px	1320px
"container-**xl**" class	100%	100%	100%	100%	1140px	1320px
"container-**xxl**"	100%	100%	100%	100%	100%	1320px
".container-**fluid**"	100%	100%	100%	100%	100%	100%

Example

The following example shows a responsive container in a body tag with content. Bootstrap 5 creates a responsive container using the "container-breakpoint" class. The default class is "container" for all web pages. If we need to use for small, medium, large, extra large and double extra large then use "container-sm", "container-md", "container-lg", "container-xl" and "container-xxl" class respectively.

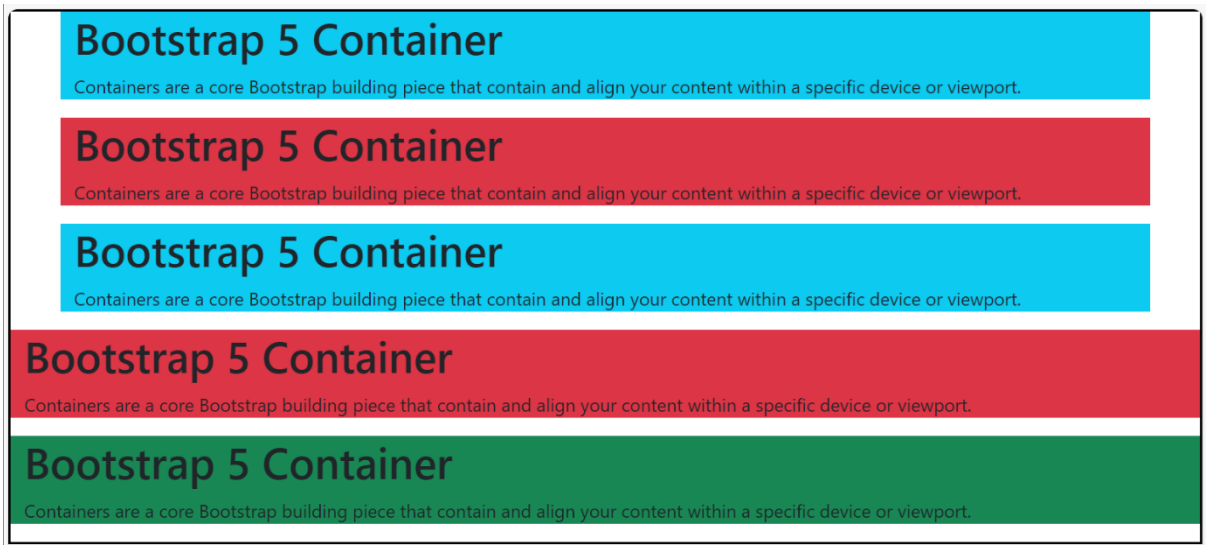
File: 2-responsive-container.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<title> Bootstrap 5 container Example </title>
<meta charset = "utf-8">
<meta name = "viewport" content = "width=device-width, initial-scale=1">
<link href = "https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel = "stylesheet">
<script src = "https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js">
</script>
<style>
body{
border:2px solid black;
}
</style>
</head>
<body>
<div class="container-sm bg-info">
    <h1> Bootstrap 5 : container-sm </h1>
    <p> Containers are a core Bootstrap building piece that contain and align your content within a specific device or viewport. </p>
</div>
<div class="container-md bg-danger ">
    <h1> Bootstrap 5 : container-md </h1>
    <p> Containers are a core Bootstrap building piece that contain and align your content within a specific device or viewport. </p>
</div>
<div class="container-lg bg-info">
    <h1> Bootstrap 5 : container-lg </h1>
    <p> Containers are a core Bootstrap building piece that contain and align your content within a specific device or viewport. </p>
</div>
```

```
<div class="container-xl bg-danger ">
    <h1> Bootstrap 5 : container-xl </h1>
    <p> Containers are a core Bootstrap building piece that contain and align you
    r content within a specific device or viewport. </p>
</div>
<div class="container-xxl bg-success ">
    <h1> Bootstrap 5 : container-xxl </h1>
    <p> Containers are a core Bootstrap building piece that contain and align you
    r content within a specific device or viewport. </p>
</div>
</body>
</html>
```

Output:

The following image shows the output of the container class example.



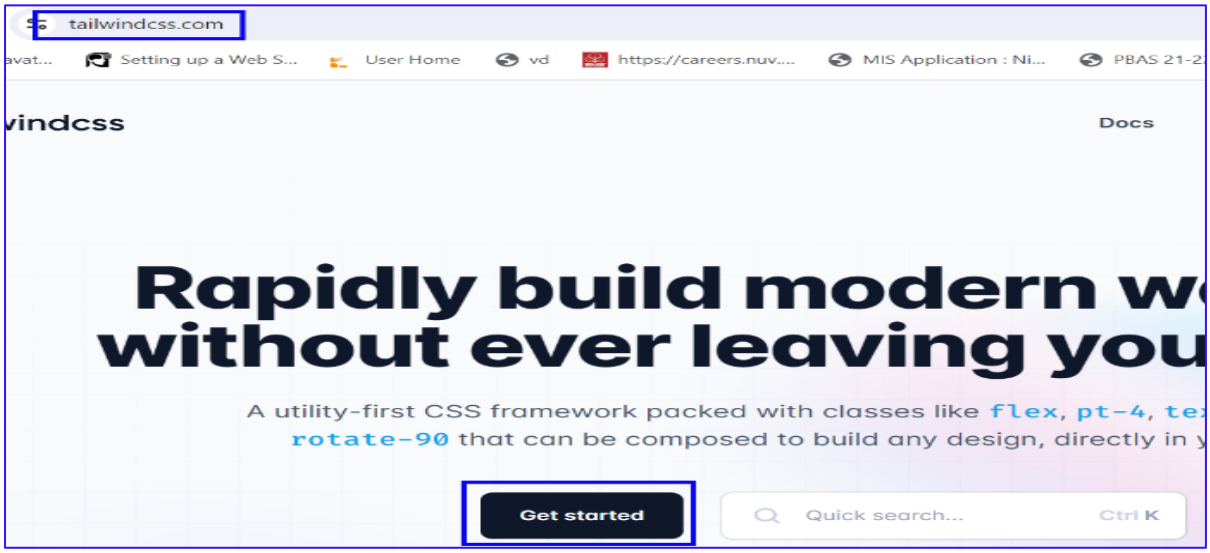
Tailwind and CSS :

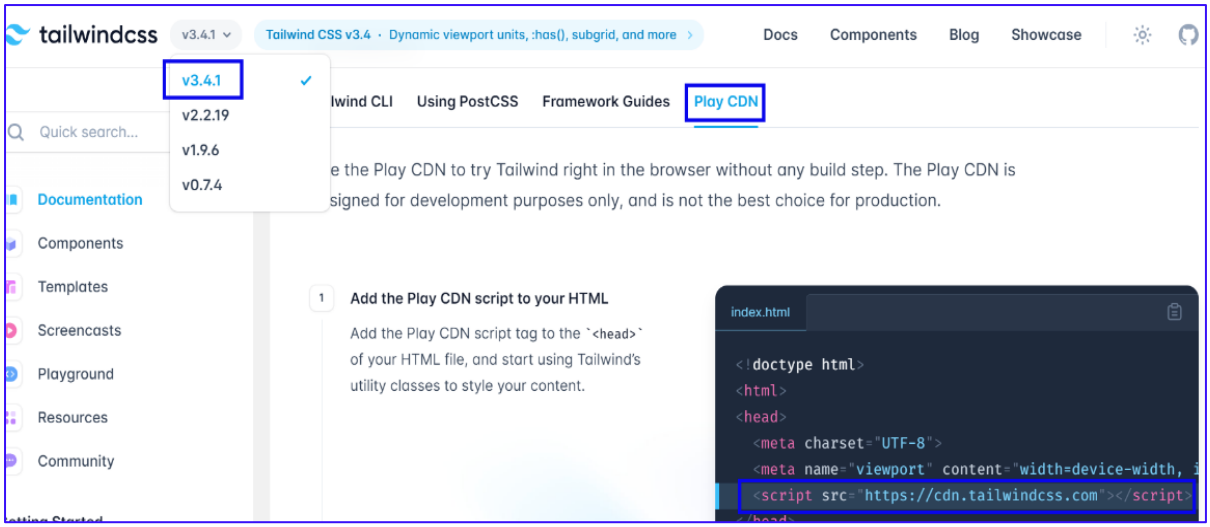
Tailwind CSS is a utility-first CSS framework that provides a set of pre-designed utility classes to build user interfaces. Unlike traditional CSS frameworks, Tailwind doesn't come with pre-built components but offers low-level utility classes that you can use to create custom designs.

Key features of Tailwind CSS include:

1. **Utility-First Approach:** Tailwind encourages the use of small, single-purpose utility classes for styling elements. This can lead to more flexibility and customization in your designs.
2. **Responsive Design:** Tailwind includes responsive utility classes, making it easy to create designs that adapt to different screen sizes.
3. **Customization:** Tailwind is highly customizable. You can use the default configuration or tailor it to your specific project needs by modifying the configuration file.
4. **Flexibility:** The framework doesn't dictate a specific design or structure, allowing you to create unique layouts and designs for your projects.
5. **Modular and Composable:** Tailwind's utility classes are modular and can be combined to create complex styles. This makes it easy to manage and understand your stylesheets.

Use the Play CDN to try Tailwind right in the browser without any build step. The Play CDN is designed for development purposes only, and is not the best choice for production.





Example-1 of Tailwind CSS in HTML: Add the Play CDN script to your HTML

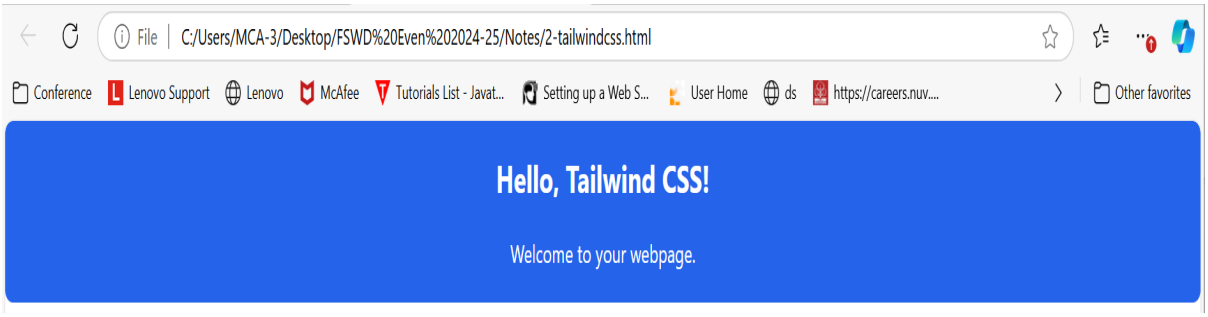
File: 2-tailwindcss.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <script src="https://cdn.tailwindcss.com"></script>
   <title>Document</title>
</head>
<body>
   <div class="bg-blue-500 text-white p-5 rounded-lg text-center">

      <h1 class="text-2xl font-bold">Hello, Tailwind CSS!</h1>
      <p class="mt-2">Welcome to your webpage.</p>

    </div>
</body>
</html>
```

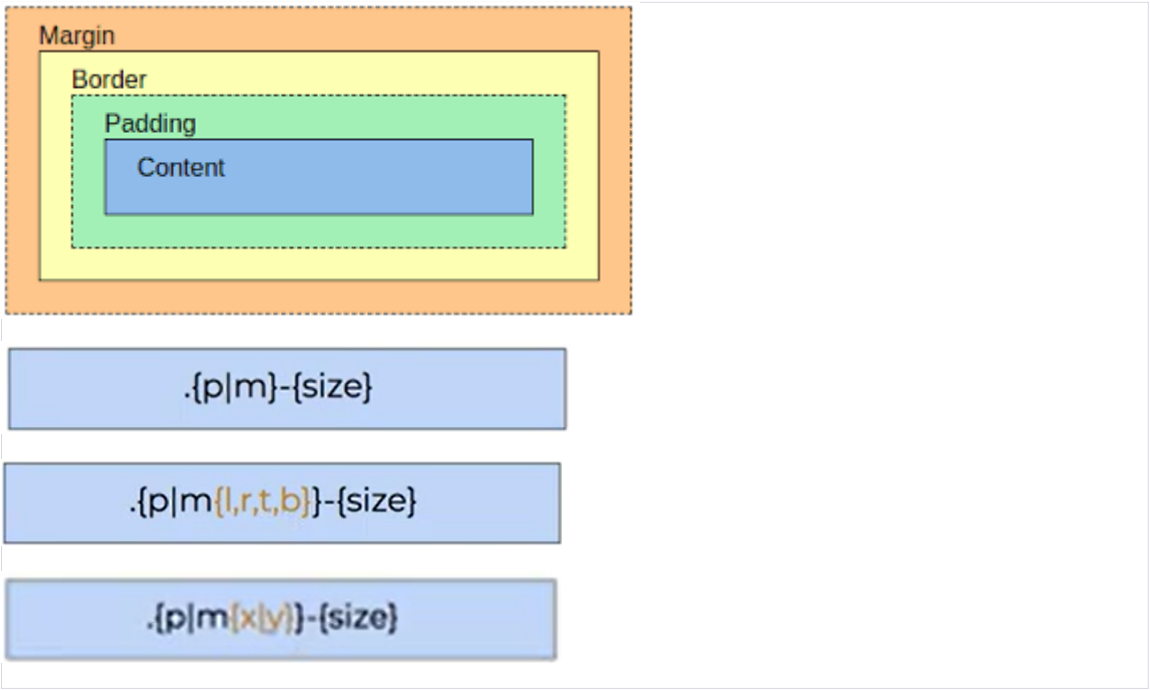
Output:



In this example, the classes like bg-blue-500, text-white, p-4, rounded-lg text-center, text-2xl, and mt-2 are Tailwind utility classes applied to style the elements.

These are examples of Tailwind CSS utility classes that apply specific styling to HTML elements. Let's break down each class:

1. bg-blue-**500**:
   - This class sets the background color of an element to a shade of blue with a specific intensity.
   - **bg** stands for background.
   - **blue** is the color, and **500** is the intensity level (500 is a common intensity scale in Tailwind).
2. text-white:
   - This class sets the text color of an element to white.
   - **text** stands for text color.
3. p-4:
   - This class adds padding to all sides (top, right, bottom, left) of an element.
   - **p** stands for padding, and **4** is the size of the padding. The scale in Tailwind is based on a default spacing scale, and 4 corresponds to a moderate padding size.
4. text-2xl:
   - This class sets the font size of text within an element to an **extra-large** size.
   - **text** stands for text-related styles, and **2xl** represents the specific size on the text scale. In Tailwind, sizes are named using various terms like **sm** (small), **md** (medium), **lg** (large), **xl** (extra-large), and so on.
5. mt-2:
   - This class adds margin to the top of an element.
   - **mt** stands for margin-top, and **2** is the size of the margin. Similar to padding, Tailwind has a scale for margin sizes.



In summary, Tailwind CSS utility classes provide a convenient way to apply styles to elements without having to write custom CSS.

They follow a naming convention that reflects their purpose and the values they apply, making it easy to understand and use them in your HTML.

Example-2 of Tailwind CSS in HTML: example of Tailwind CSS that describes how to change the background color on mouse hover.

```
<!DOCTYPE html>
<html lang="en">

<head>
        <meta charset="UTF-8">
        <!-- Tailwind CSS CDN link -->
    <script src="https://cdn.tailwindcss.com"></script>
</head>

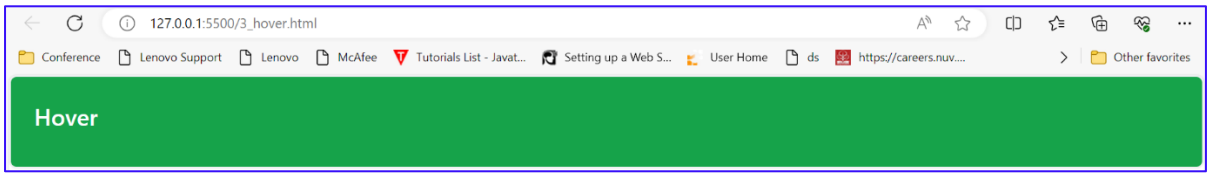
<body>
        <div class="h-full border-2 border-gray-200
                                border-opacity-60 rounded-lg
                                overflow-hidden">
```

```
            <div class="p-6 hover:bg-green-600
                        hover:text-white
                    transition duration-300 ease-in">

                    <h1 class="text-2xl font-semibold mb-3">
                        Hover
                    </h1>
            </div>
        </div>
</body>

</html>
```

Output:



This code is an HTML document that utilizes Tailwind CSS, a popular utility-first CSS framework.

1. `<script src="https://cdn.tailwindcss.com"></script>`: This script tag imports the Tailwind CSS library from the specified CDN, allowing the document to utilize Tailwind CSS classes.

2. `<div>` element has several Tailwind CSS classes applied to it:

- h-full: This class sets the height of the `<div>` to be equal to the full height of its parent container. This ensures that the `<div>` takes up all available vertical space.
- border-2: This class adds a border to the `<div>` with a thickness of **2** pixels.
- border-gray-200: This class specifies the color of the border to be a shade of gray (specified by the Tailwind CSS color palette).

- border-opacity-60: This class sets the opacity of the border to **60**%. This means that the border will be somewhat transparent.
- rounded-lg: This class rounds the corners of the <**div**>, giving it a visually softer appearance. The "lg" in **rounded-lg** stands for "large," indicating that the rounded corners will be relatively large.
- overflow-hidden: This class hides any content that overflows the boundaries of the <**div**>. It ensures that any content extending beyond the size of the <**div**> will not be visible and will be clipped instead.

In summary, this <**div**> element is styled using Tailwind CSS to have a full height, a 2-pixel gray border with 60% opacity and rounded corners, and it hides any content that overflows its boundaries.

3. <**div**> element has several Tailwind CSS classes applied to it:

- p-6: This class adds padding to the <**div**> on all sides. Specifically, it adds padding of size 6 inside the <**div**> content area.
- hover:bg-green-600: This class specifies a background color change when the <**div**> is hovered over. Specifically, it changes the background color to green with a shade represented by **600** in the Tailwind CSS color palette. The **hover:** prefix indicates that this style is applied only when the <**div**> is being hovered over.
- hover:text-white: This class specifies a text color change when the <**div**> is hovered over. Specifically, it changes the text color to white. Again, the **hover:** prefix indicates that this style is applied only when the <**div**> is being hovered over.
- transition: This class specifies a transition effect for the <**div**> when its state changes. It's used in conjunction with other classes like **hover:bg-green-600** and **hover:text-white** to smoothly animate the background color and text color changes.
- duration-300: This class specifies the duration of the transition effect in milliseconds. In this case, the transition duration is set to 300 milliseconds, meaning the color change will occur gradually over a period of 0.3 seconds.

- ease-in: This class specifies the timing function for the transition effect. The **ease-in** timing function starts slowly and accelerates gradually. It provides a smooth transition effect, making the color change appear more natural.

In summary, this ** ** element is styled using Tailwind CSS to have padding, change its background color and text color on hover, and apply a smooth transition effect to these changes.

Vue JS: Directives and Binding

What is Vue.js?

Vue.js is an **open-source progressive (**offer variable degrees of complexity and flexibility**) JavaScript framework** used to develop interactive **web user interfaces** and **single-page applications (SPAs)**.

Vue.js is commonly referred to as Vue and pronounced as "**view.js**" or "**view**."

- Vue.js lets you **extend** HTML with HTML attributes called **directives**
- Vue.js directives offers **functionality** to HTML applications
- Vue.js provides **built-in** directives and **user defined** directives

What is a Single Page Application (SPA)?

A single page application or **SPA** is a web application or a website that provides users a very fluid, reactive, and fast experience similar to a desktop application.

A single page application contains a menu, buttons, and blocks on a single page.

When a user clicks on any of them, it dynamically rewrites the current page rather than loading **entire new pages from a server**. That's the reason behind its reactive fast speed.

Vue is basically developed for frontend development, so it has to deal with a lot of HTML, JavaScript, and CSS files. Vue.js facilitates users to extend HTML

with HTML attributes called directives. Vue.js provides built-in directives and a lot of user-defined directives to enhance functionality to HTML applications.

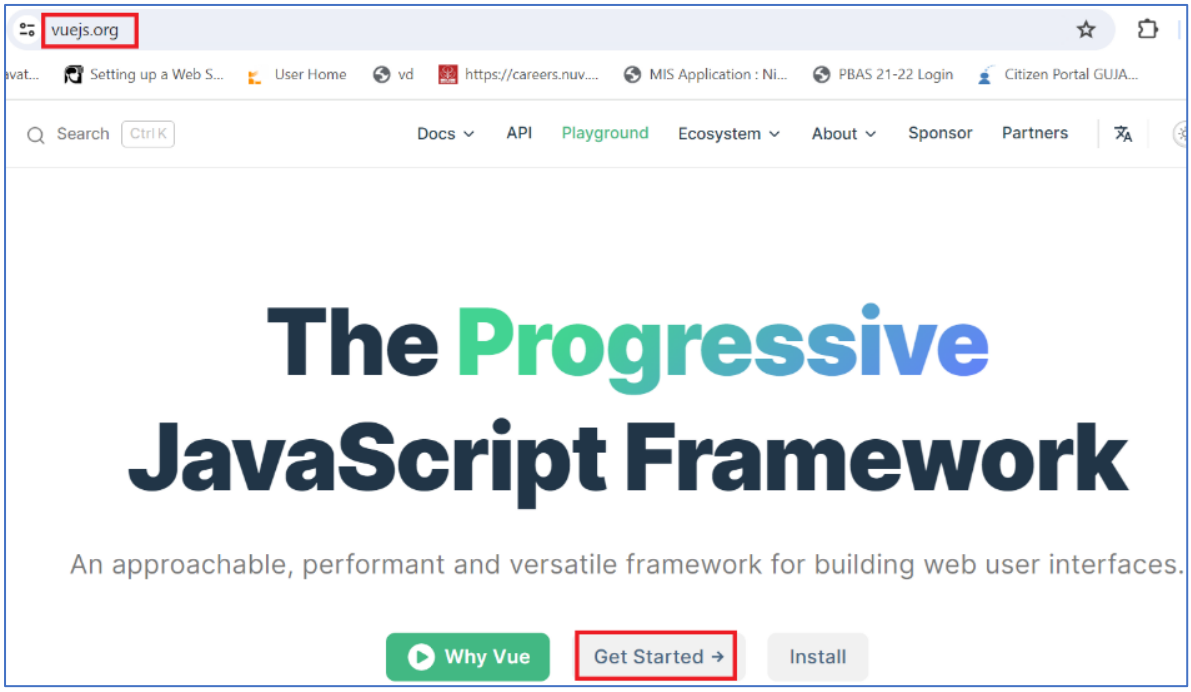
Pre-requisites:

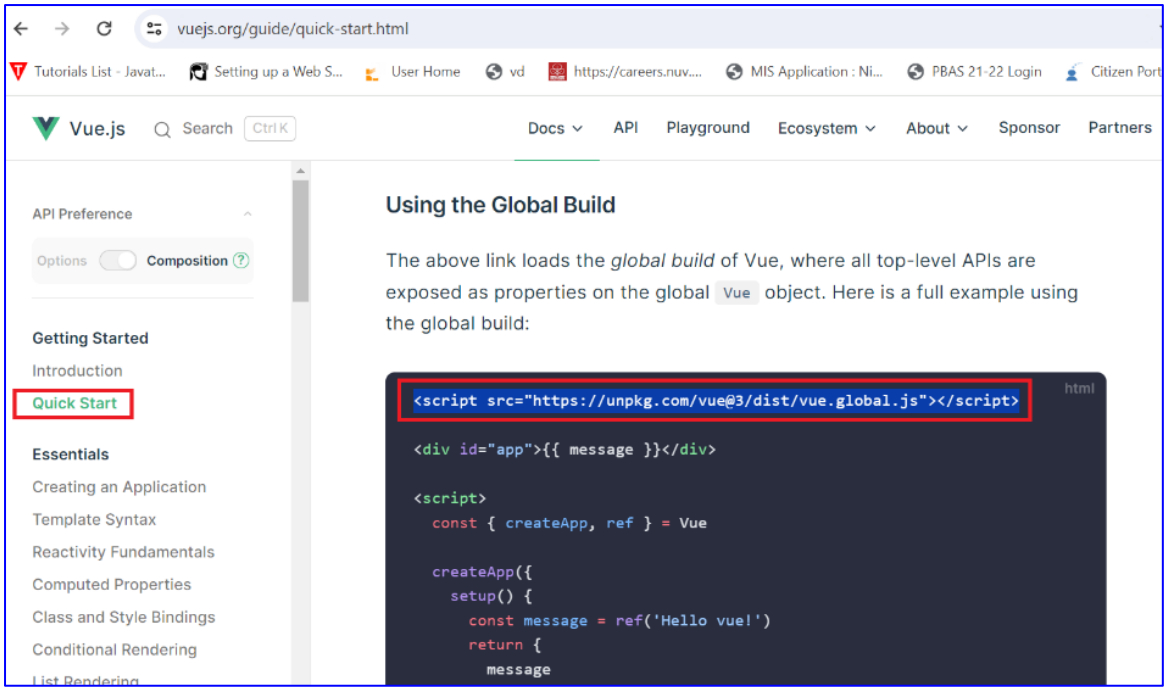
- Install Node.js version 15.0 or higher
- Text Editor/Code Editor — Visual Studio Code, Notepad++, Atom etc
- Web Browser — Google Chrome, Firefox
- Volar Extension If VS Code
- Vue.js devtools a Chrome Extension

https://vuejs.org/guide/scaling-up/tooling.html

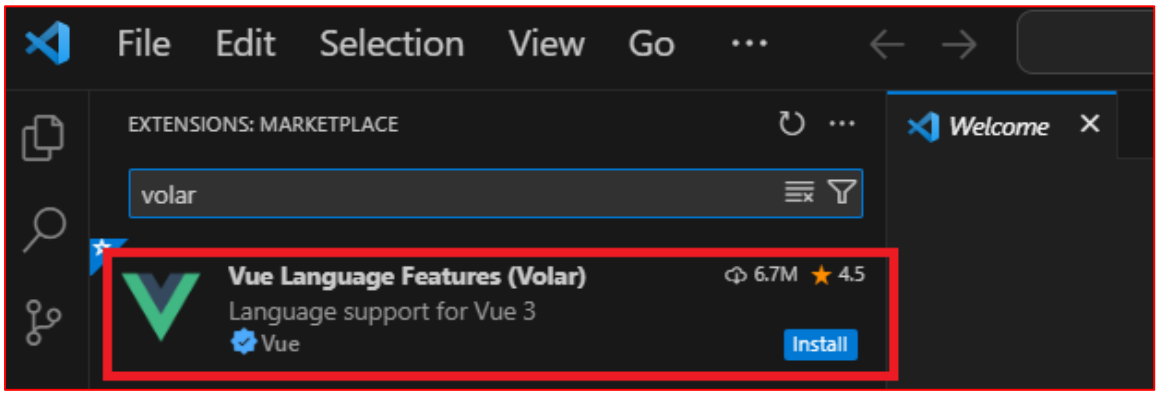
Create Prject:

A. Without build tool (CDN)









File Name: index.html (How is the Vue.js framework integrated into the provided HTML file?)

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Vue JS - FSWD </title>
   <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
   <h1>Hello F-D Division</h1>
   <div id="app">{{ message }}</div>

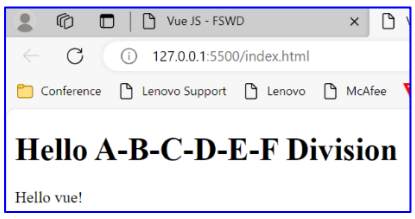
    <script>

     const { createApp, ref } = Vue

     createApp({
       setup()
       {
         const message = ref('Hello vue!')
         return { message }
       }
     }).mount('#app')

    </script>
</body>
</html>
```

Ooutput:



The above code snippet demonstrates how to create a simple Vue.js application using the Composition API, which was introduced in Vue.js 3.0. brief explanation of each part:

1. **HTML Structure**: The HTML structure consists of a **<body>** element containing an **<h1>** heading with the text "Hello F-D Division", and a ** ** element with the **id** attribute set to "app". Inside the ** **, there's an interpolated expression **{{ message }}**, which indicates that it will be replaced with the value of the **message** variable from Vue.js.
2. **Vue.js Setup**: In the **<script>** tag, the code imports **createApp** and **ref** from Vue.js.
3. **Vue App Creation**: The **createApp** function is called with an object as an argument. This object represents the configuration of the Vue.js application.
4. **Setup Function**: Within the configuration object, there's a **setup** function. The **setup** function is a special function in Vue 3 that is executed before the component is created. It's used for setting up reactive properties and other logic. Inside the **setup** function:
    - A reactive property **message** is created using the **ref** function. The initial value of **message** is set to 'Hello vue!'.
    - The **message** variable is returned from the **setup** function, making it accessible within the Vue component.
5. **Mounting the App**: The **createApp().mount('#app')** method is called to mount the Vue.js application onto the DOM element with the **id** of "app". This connects the Vue instance to the HTML element with the specified ID, allowing Vue.js to take control of the element and its children.
6. **Rendering**: Vue.js will now render the template associated with the Vue instance. Since the template contains **{{ message }}**, Vue.js will replace **{{ message }}** with the current value of the **message** reactive property. Initially, it will display "Hello vue!".

So, the execution flow involves creating a Vue.js application, defining reactive properties within the **setup** function, and mounting the application onto the DOM element with the specified ID. Finally, Vue.js renders the template, replacing any template expressions with the current values of the associated reactive properties.

In summary, this code sets up a simple Vue application with a single reactive variable **message**, initialized with the value **'Hello vue!'**, and mounts it to an HTML element with the id **#app**.

OR

File Name: index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Vue JS - FSWD </title>
   <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
   <h1>Hello F-D Division</h1>
   <div id="app">{{ message }}</div>

<script src='./myscript.js'></script>
</body>
</html>
```

File Name: myscript.js

```
const { createApp, ref } = Vue


 createApp({
   setup() {
     const message = ref('Hello vue!')
     return {message}
   }
 }).mount('#app')
```

B. (Practical-7 : custom directive):

A **custom directive** in Vue.js allows you to create reusable behaviors that can be applied to DOM elements.

Vue has built-in directives like v-if, v-for, and v-model, custom directives let you define your own logic and apply it to elements.

Creating a Custom Directive

You can create a custom directive globally or locally.

Demonstrate how to create a custom directive and how to use it through HTML web pages.:

I. Create a custom directive to transform text to uppercase when clicked.
II. Creating a Dynamic List with Vue.js Custom Directive
III. Create a directive that formats and displays dates in a human-readable format

---

Step-1: Define the Directive: Use `Vue.directive()` to define your custom directive. You need to specify a name for the directive and an object containing hook functions that define the directive's behavior.

Step-2: Hook Functions: There are several hook functions you can use to define the behavior of your directive. The most commonly used ones are:

- **`bind`**: Called only once when the directive is first bound to the element. You can perform one-time setup here.
- **`inserted`**: Called when the bound element is inserted into the DOM. This is where you can perform DOM manipulation.
- **`update`**: Called when the bound element's value or expression changes.
- **`componentUpdated`**: Called after the component containing the bound element has been updated.
- **`unbind`**: Called only once when the directive is unbound from the element.

Step-3: Accessing the Element and Data: Within the hook functions, you can access the element the directive is bound to, as well as any data passed to the directive.

I. Create a custom directive to transform text to uppercase when clicked.

File: capitalize.html

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Custom Directive Example</title>
 <script src="https://cdn.jsdelivr.net/npm/vue@2"></script>
</head>
<body>

<div id="app">
 <p v-uppercase>Hello, this text will be uppercased when clicked!</p>
</div>

<script src="app.js"></script>
</body>
</html>
```

`<p v-uppercase>`: This is a paragraph element (`<p>`) to which a Vue directive named `v-uppercase` is applied. Directives in Vue.js are special attributes prefixed with `v-`, which allow you to apply reactive behavior to the DOM. In this case, `v-uppercase` is a custom directive that will be triggered when the paragraph is interacted with.

File: app.js

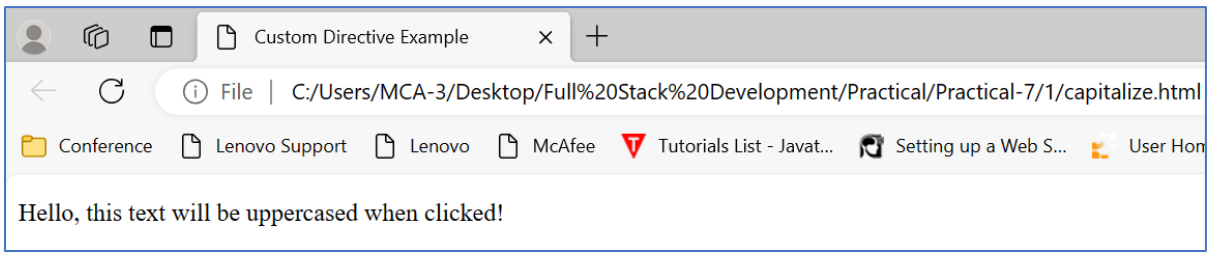
```
Vue.directive('uppercase', {
 bind(el) {
  el.style.cursor = 'pointer'; // Changes the cursor style to pointer when hovering over the element

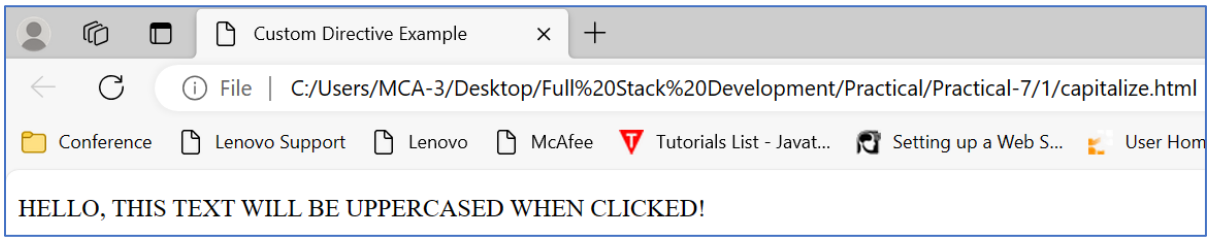
  el.addEventListener('click', () => { // Attaches a click event listener to the element
    const text = el.innerText; // Retrieves the text content of the element
```

```
    el.innerText = text.toUpperCase(); // Changes the text content to
uppercase
    });
  }
});

// Create a new Vue instance
new Vue({
  el: '#app' // Mounts the Vue instance to the element with the ID 'app'
});
```

Output:





Here's what each part of the code does:

1. `Vue.directive('uppercase', { ... })`: This line registers a custom directive named `'uppercase'` with Vue.js. Custom directives in Vue.js allow you to extend the behavior of HTML elements. In this case, the directive will make the text content of an element uppercase when clicked.
2. `bind(el) { ... }`: This is the definition of the directive's **hook function**. The `bind` hook is called only once when the directive is

first bound to the element. It receives the element (`el`) as its argument.

3. `el.style.cursor = 'pointer';`: This line sets the cursor style of the element to `pointer`, making it appear as a hand icon when hovered over. This is a visual cue to the user that the element is clickable.
4. `el.addEventListener('click', () => { ... });:` This line attaches a click event listener to the element. When the element is clicked, the callback function inside the event listener will be executed.
5. `const text = el.innerText;:` This line retrieves the text content of the element and stores it in the `text` variable.
6. `el.innerText = text.toUpperCase();:` This line sets the text content of the element to its uppercase version. It uses the `toUpperCase()` method to convert the text to uppercase.
7. `new Vue({ el: '#app' });:` This code creates a new Vue instance and mounts it to the element with the ID `app`. The Vue instance is responsible for managing the behavior and data of the application within the specified element.

Overall, this code defines a custom Vue directive named `'uppercase'` that makes the text content of an element uppercase when clicked. It also changes the cursor style of the element to indicate that it is clickable.

II. Creating a Dynamic List with Vue.js Custom Directive

File: index.htlm

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Custom Directive Example</title>
 <script src="https://cdn.jsdelivr.net/npm/vue@2"></script>
```

```
</head>
<body>

<div id="app">
  <ul v-list="items"></ul>
</div>

<script src="app.js"></script>
</body>
</html>
```

File: app.js

```
Vue.directive('list', {
  bind(el, binding) {
    const ul = document.createElement('ul'); // Create a new <ul> element

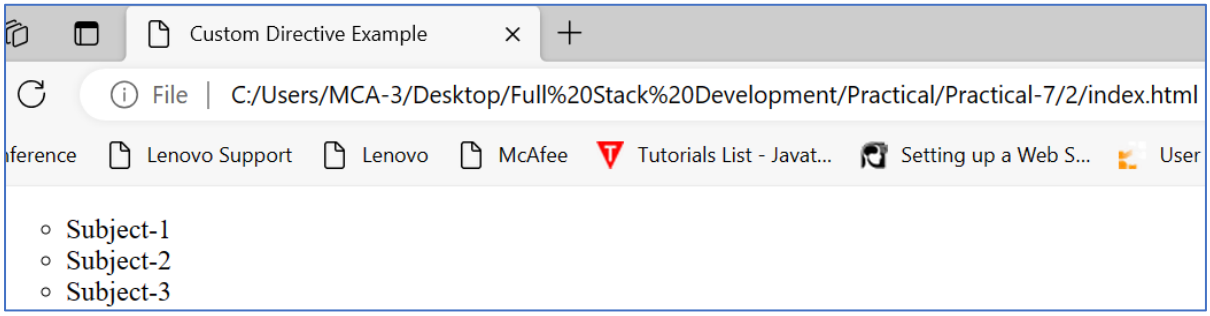
    binding.value.forEach(item => { // Iterate over each item in the binding value
      const li = document.createElement('li'); // Create a new <li> element
      li.textContent = item; // Set the text content of the <li> element to the current item
      ul.appendChild(li); // Append the <li> element to the <ul> element
    });
    el.appendChild(ul); // Append the <ul> element to the bound element (the element the directive is applied to)
  }
});

// Create a new Vue instance

new Vue({
  el: '#app', // Mount the Vue instance to the element with the ID 'app'
  data: {
    items: ['Subject-1', 'Subject-2', 'Subject-3','Subject-4'] // Sample list of items
```

```
 }
});
```

Output:



III. Create a directive that formats and displays dates in a human-readable format

File: v-format.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Custom Directive for Date Formatting</title>
  <script src="https://cdn.jsdelivr.net/npm/vue@2"></script>
</head>
<body>

<div id="app">
  <p v-format-date="date"> <!-- Apply the custom directive -->
    <!-- The content will be manipulated by the custom directive -->
  </p>
</div>

<script>
// Define the custom directive named 'format-date'
Vue.directive('format-date', {
  // When the bound element is inserted into the DOM...
```

```
  inserted: function(el, binding) {
    // Get the date from the binding value
    const date = binding.value;

    // Format the date into a human-readable format
    const formattedDate = formatDate(date);

    // Update the element's content with the formatted date
    el.textContent = formattedDate;
  }
});

// Create a new Vue instance
new Vue({
  el: '#app',
  data: {
    date: '2022-03-05T12:00:00' // Sample date
  }
});

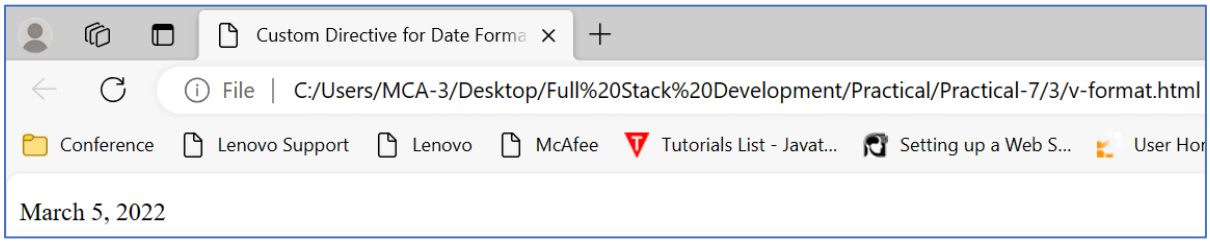
// Helper function to format the date
function formatDate(dateString)
{
  const options = { year: 'numeric', month: 'long', day: 'numeric' };
  return new Date(dateString).toLocaleDateString('en-US', options);
}
</script>

</body>
</html>
```

This line converts the input `dateString` into a JavaScript `Date` object using `new Date(dateString)`. Then, it uses the `toLocaleDateString` method to format the date based on the provided options (`options`

object). The first argument `'en-US'` specifies the locale to be used for formatting the date.

Output:



React JS: Elements, UI Components, JSX, Redux, Fetch Data and Asynchronous Programming, GraphQL APIs

React JS:

React Introduction

Currently, ReactJS is one of the most popular JavaScript front-end libraries which has a strong foundation and a large community.

ReactJS is a **declarative(**Tell react what you want and react will build the actual UI**)**, **efficient**, and flexible **JavaScript library** for building reusable UI components.

It is an open-source, component-based front-end library which is **responsible only for the view layer of the application.**

It was initially developed and maintained by Facebook and later used in its products like WhatsApp & Instagram.

A ReactJS application is made up of multiple **components**, each component responsible for **outputting** a small, reusable piece of HTML code. The components are the heart of all React applications. These Components can be nested with other components to allow complex applications to be built of simple building blocks.

ReactJS uses **virtual DOM** based mechanism **to fill data in HTML DOM**. The virtual DOM works fast as it only changes individual DOM elements instead of reloading complete DOM every time.

To create React app, we write React components that correspond to various elements. We organize these components inside higher level components which define the application structure.

For example:
we take a **form** that consists of many elements like input fields, labels, or buttons. We can write each element of the form as React components,
and then we **combine** it into a higher-level component, i.e., the form component itself.
The form **components would specify the structure** of the form along with elements inside of it.

Prerequisite

Before learning ReactJS in-depth, you must have a good knowledge of JavaScript, HTML5, and CSS.

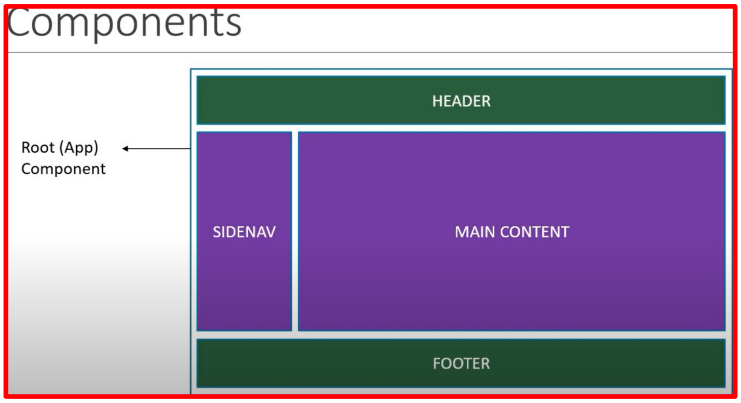
Difference between Virtual DOM and Real DOM

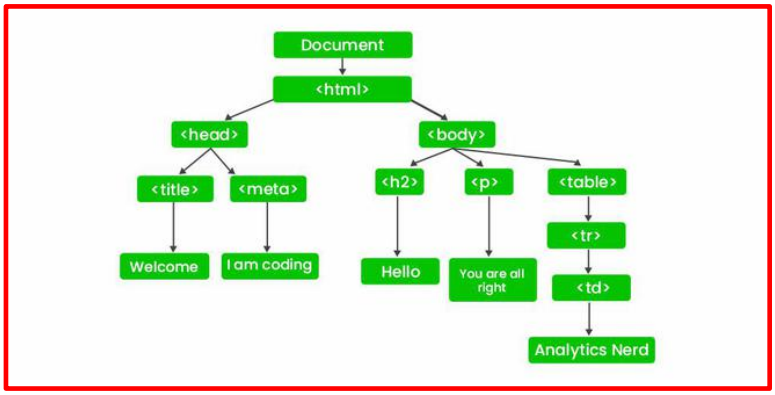
DOM stands for **D**ocument **O**bject **M**odel, it is the structural representation of the HTML Document. **Real** DOM is the actual structure represented in the User Interface while **Virtual** DOM is the memory representation of the same.

It (**Real** DOM) is a tree-like Structure consisting of all nodes in an HTML document DOM represents the Ul of your applications.

Real DOM

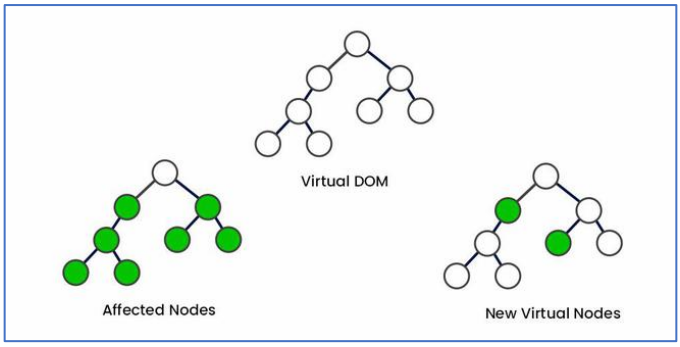
- Real DOM is the actual structure of the webpage.
- React Update complete document in the Real DOM.
- React DOM is the actual webpage rendered on the browser any changes made directly reflect on the complete webpage.
- The DOM represents the web page often called a document with a logical tree and each branch of the tree ends in a node and each node contains objects.
- The developer can modify the content of the document using a scripting language like JavaScript.
- The changes and updates to the DOM are fast because of its tree-like structure **but** re-rendering whole documents makes the DOM Slow.
- All UI components need to be re-rendered for every DOM update.





Virtual DOM

- Virtual DOM is the virtual representation of Real DOM
- React update the **state** changes in Virtual DOM first and then it syncs with Real DOM
- Virtual DOM is just like a blueprint of a machine, can do changes in the blueprint but those changes will not directly apply to the machine.
- Virtual DOM is a programming concept where a virtual representation of a UI is kept in memory synchronized with "Real DOM " by a library such as ReactDOM and this process is called reconciliation
- Virtual DOM makes the performance faster, not because the processing itself is done in less time. The reason is the amount of changed information – rather than wasting time on updating the entire page, you can separate it into small elements and interactions



Why we use ReactJS?

The **main objective** of ReactJS is to develop User Interfaces (UI) that improves the speed of the apps. It uses virtual DOM (JavaScript object), which improves the performance of the app. The JavaScript virtual DOM is faster than the regular DOM.

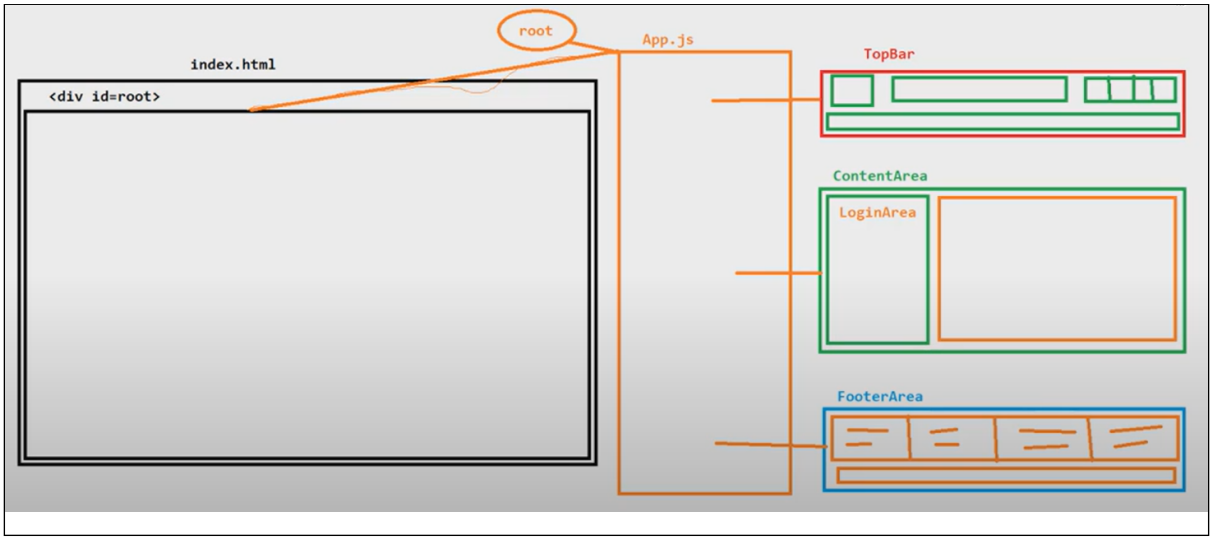
We can use ReactJS on the client and server-side as well as with other frameworks. It uses component and data patterns that improve readability and helps to maintain larger apps.

Today, many JavaScript frameworks are available in the market(like angular, node), but still, React came into the market and gained popularity amongst them.

The **previous frameworks** follow the traditional data flow structure, which uses the DOM (Document Object Model). DOM is an object which is created by the browser each time a web page is loaded. It dynamically adds or removes the data at the back end and when any modifications were done, then each time a new DOM is created for the same page. This repeated creation of DOM makes unnecessary memory wastage and reduces the performance of the application.

Therefore, a new technology ReactJS framework invented which remove **above drawback**. ReactJS allows you to divide your entire application into various components. ReactJS still used the same traditional data flow, but it is not directly operating on the browser's Document Object Model (DOM) immediately; instead, it operates on a virtual DOM. It means **rather** than manipulating the document in a browser after changes to our data, it resolves changes on a DOM built and run entirely in memory. After the virtual DOM has been updated, React determines what changes made to the actual browser's DOM.

The React Virtual DOM exists entirely in memory and is a representation of the web browser's DOM. Due to this, when we write a React component, we did not write directly to the DOM; instead, we are writing virtual components that react will turn into the DOM.



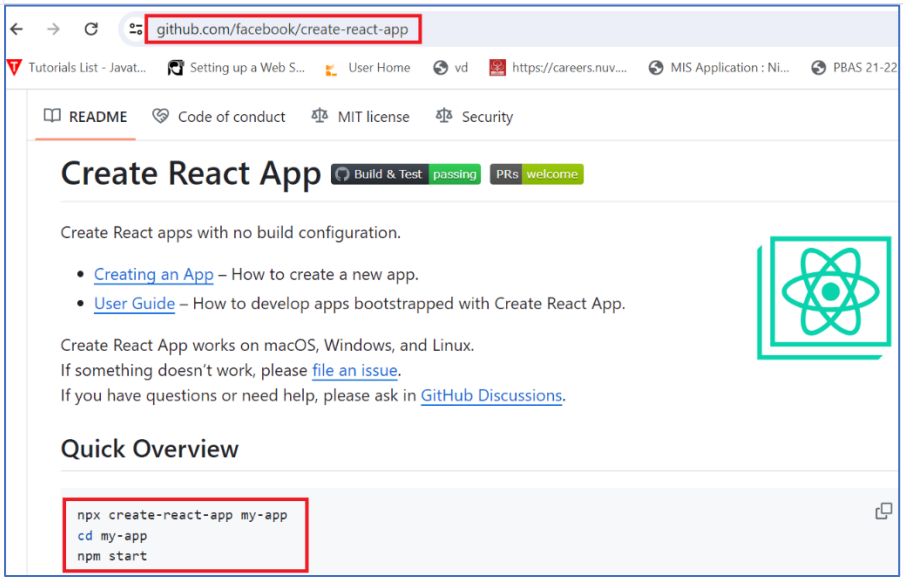
To install React.js, you need to follow these steps:

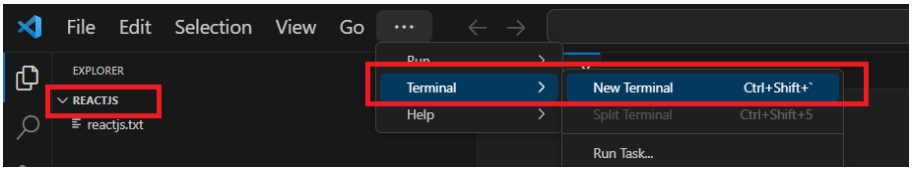
1. **Setup Node.js and npm**:

   React.js relies on Node.js and npm (Node Package Manager) for package management. If you haven't already installed Node.js and npm, you can download and install them from the official Node.js website: https://nodejs.org/. Follow the installation instructions for your operating system.

2. **Create a New React Project**:

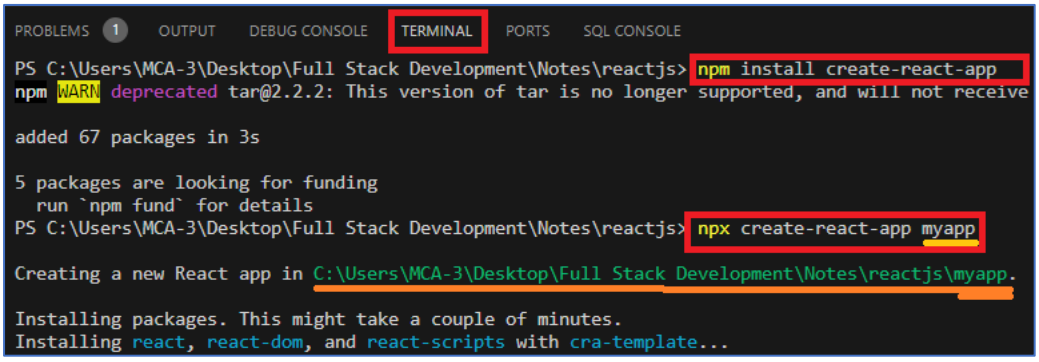
   Once you have Node.js and npm installed, you can create a new React project using Create-React-App. Create React App is a tool built by Facebook for easily setting up a new React project with a predefined folder structure and build configuration.
   To create a new React project, open your terminal and run the following command:





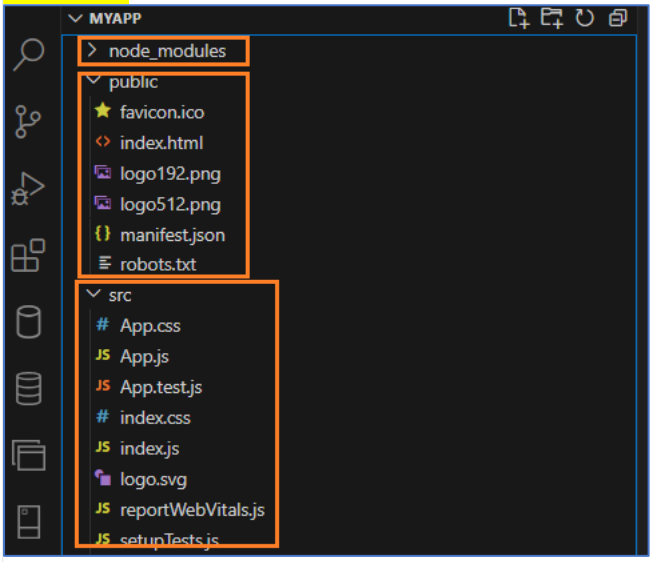
If npx create-react-app hello-world doesn't work or generates an error then
To fix the issue, run these commands in sequence:

```
npm init
npm install create-react-app
npx create-react-app myapp
```



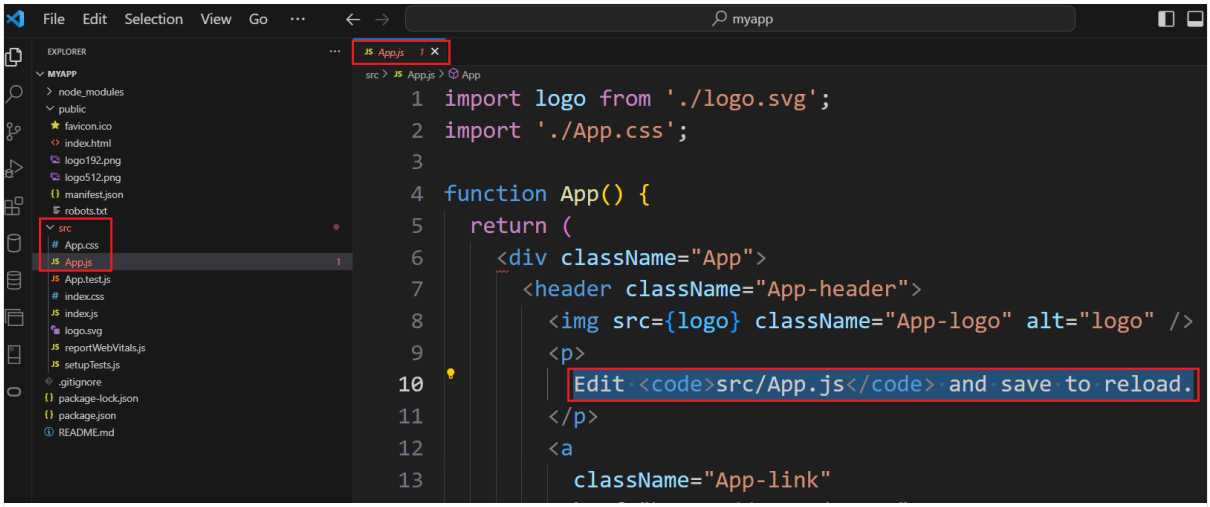
3. **Navigate to Project Directory:** Once the project is created, navigate to the project directory using the following command:

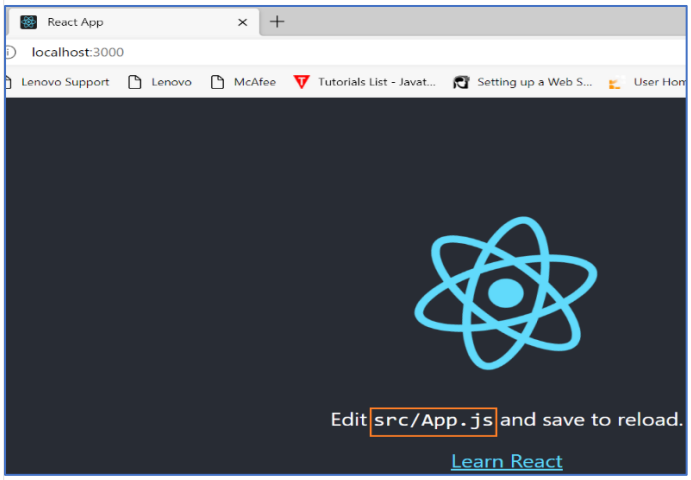
```
cd myapp
```



4. **Start the Development Server:** After navigating to the project directory, you can start the development server by running the following command:

```
npm start
```







This command will start a development server and open your default web browser to display your React application. Any changes you make to your React components will automatically be reflected in the browser.

5. **Explore and Edit**: Now that your React project is set up and running, you can start exploring the project structure and editing the files. The **src** directory contains

all your React components, and you can begin modifying them to build your application.

This is how, successfully installed React.js and set up a new React project. You can continue building your application by adding components, managing state, and handling user interactions.

React JS: Elements

Introduction to React Element:

Here, we will understand about **what** are React elements, **how** do we create Elements and **how** do we render Elements

Elements are the **smallest** building blocks of React apps. An element specifies **what** should be there in our UI. An Element is a plain object describing **what** we want to **appear** in terms of the DOM nodes.

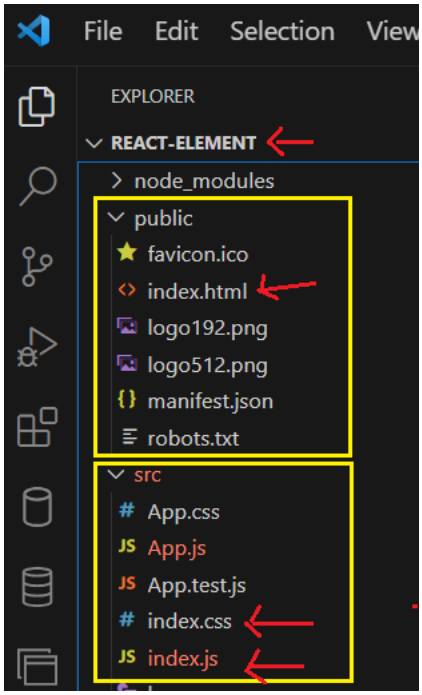
Creating a React Element is Cheap compared to DOM elements. An Element can be Created by using JSX **or** React without JSX.

Let's create React Element using JSX.

```
npm init
npm install create-react-app
```

Step-1: create a project by using the below command.

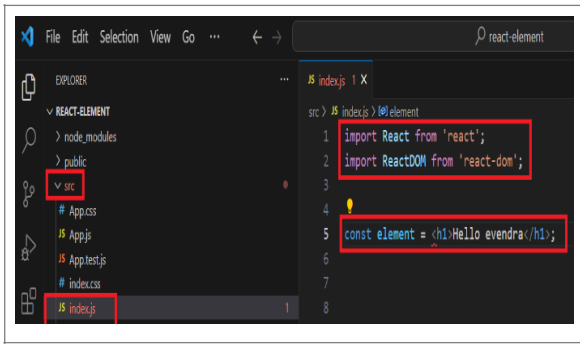
```
npx create-react-app REACT-ELEMENT
```

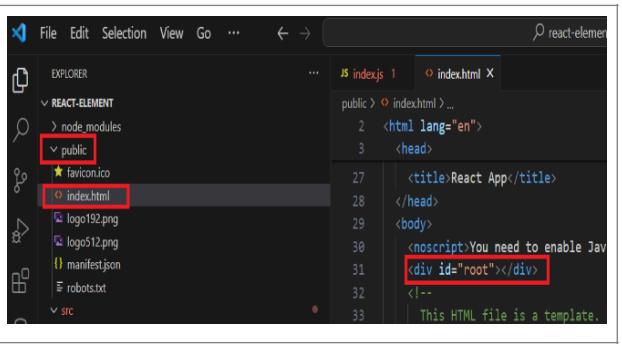


Step-2: open **index.js** from the **src** folder and

open **index.html** from the **public** folder

We have created an Object of type h1 and assigned it to a variable called as element. This element should be rendered into the Browser DOM, and for that we need a container. Open Index.html and there is a div with id as root and we will use that div as our container.





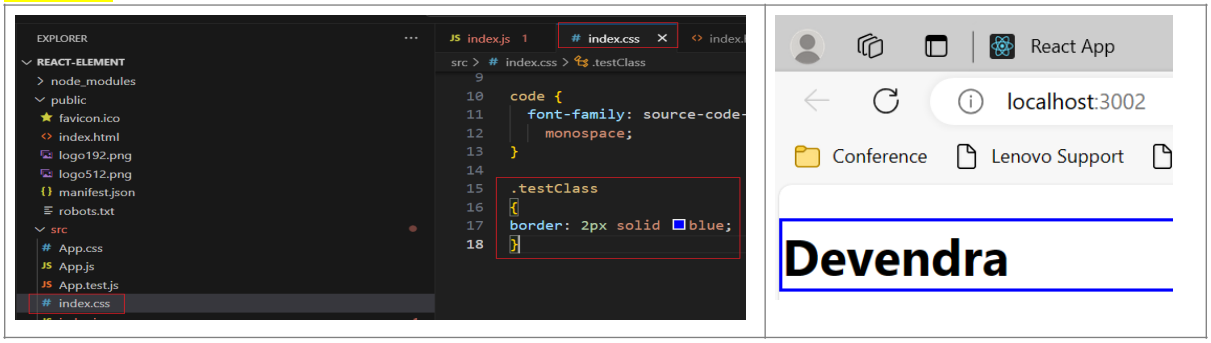
Step-3: The above created Element can be rendered into it By writing statement as bellow:

```
index.js 1 ●   index.html
src > index.js > ...
1  import React from 'react';
2  import ReactDOM from 'react-dom';
3
4
5  const element = <h1>Hello evendra</h1>;
6
7
8  ReactDOM.render(element, document.getElementById('root'));
```

import React from 'react'; =>
This imports the React library, which is required to create React components.
import ReactDOM from 'react-dom'; =>
This imports the ReactDOM library, which is used to render React components into the actual DOM.

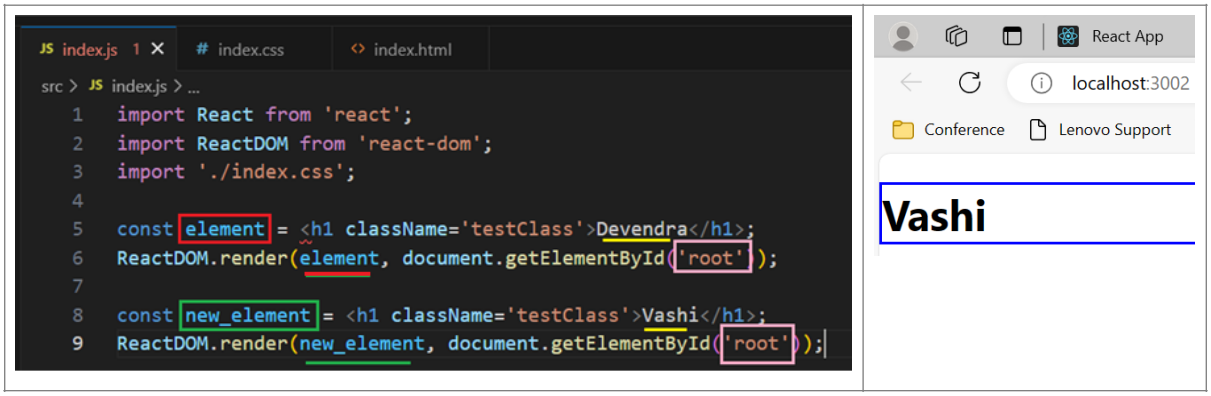
Step-4:

Lets say we want to assign some styles to this element. Create a css class with classname as testclass in index.css. Apply this class to the element using className attribute.



Step-5:

An Element contains type and properties, here h1 is the type and this element contains className as property. An Element can be as simple as the one we have created or an element can contain elements inside it.



The above code we have written using JSX would be compiled into Plain Javascript using Babel.(**Babel** is a popular JavaScript compiler that can transform JSX code into plain JavaScript code that browsers can execute.)

React JS: Function Components

Introduction to Components in React

Here, we will understand about **what** are React Components, **how** do we create Components and **how** do we render Components.

Components are the building blocks of any React app. Components allow us split the UI into independent and reusable pieces.

A component is combination of

1. Template using HTML

2. User Interactivity using JS

3. Applying Styles using CSS

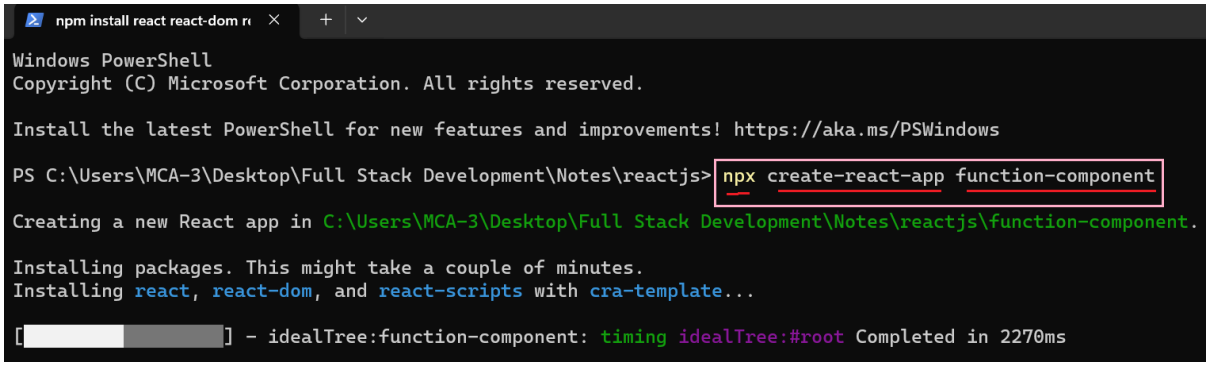
A typical React app will have **many components like header** component, **navbar** component, **footer** component and **content** component.

Conceptually a component is a JavaScript class or function that accepts inputs which are properties(props) and returns a React element that describes how a section of the User Interface should appear. A component can be created as Function Component or Class Component.

Function component:

Open the **function-component**(project title in VS code) using VS code and run this Project using Nodejs Command Prompt.

Lets say we want to Create a Javascript function which takes an Object and returns a container which displays Employee Information.



import React from 'react'; =>
This imports the React library, which is required to create React components.
import ReactDOM from 'react-dom'; =>
This imports the ReactDOM library, which is used to render React components into the actual DOM.



The above function (arrow function) is an example of Component and we can Call this by writing

```
const element = <Employee name="Sara" salary="12345"
dept="CSE" head="Ahmedabad" />;
```

and this element can be rendered using

```
ReactDOM.render(element,  document.getElementById('root'));
```

When React sees an element representing a user-defined component, it passes JSX attributes to this component as a single object. We call this object "props".

The components we have created until now are called as Function Components.

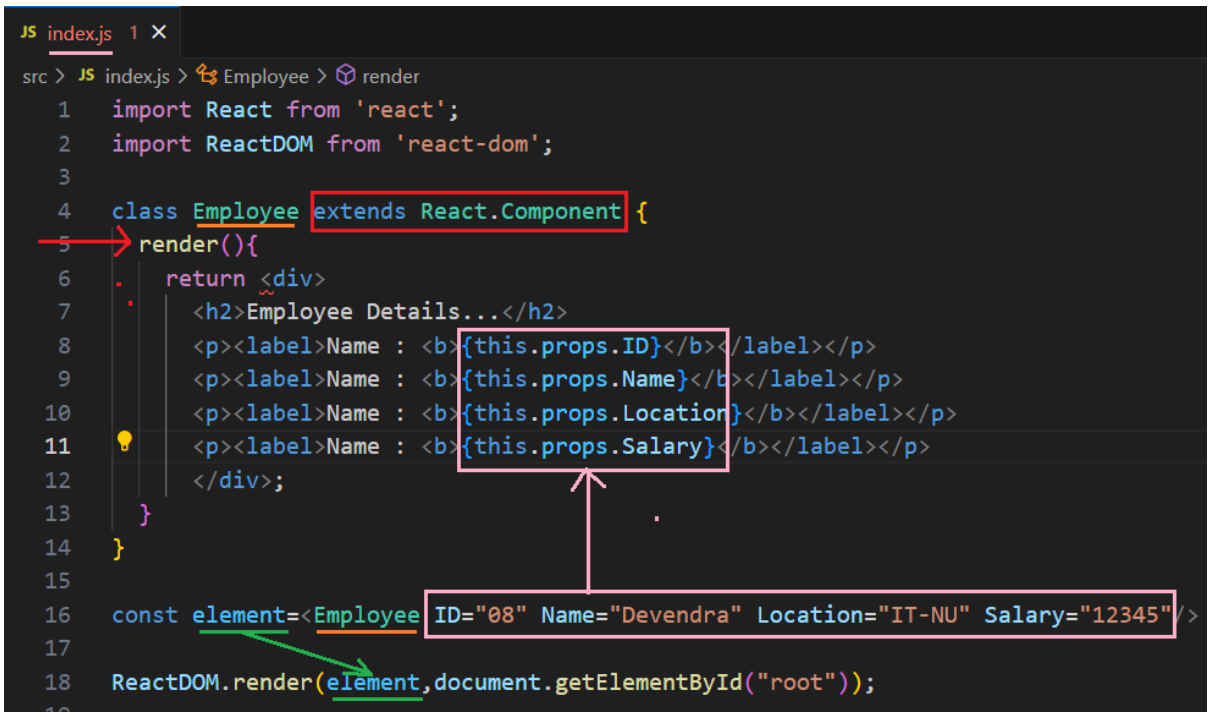
React JS: CLass Components

npx create-react-app class-component

To define a React component class, We have to create a class and extend React.Component class. Lets say we want to create an Employee Component and this Component should return the Element which displays Employee Details.

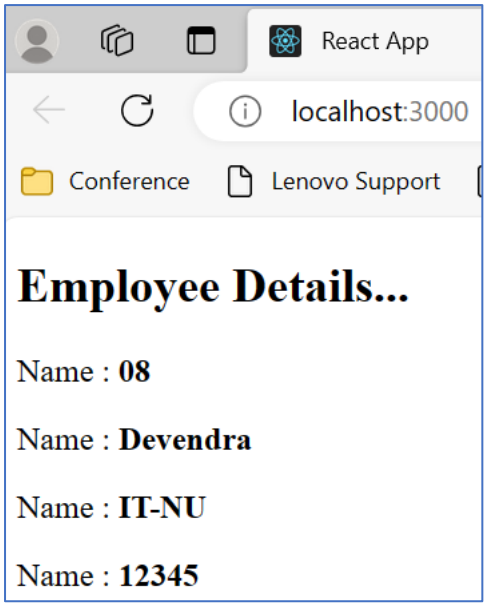
Open Index.js file from our Demo-Project, Create Employee class and extend it from React.Component. Output of any Class Component we create is dependent on the return value of a Method Called **render().** The **render**() method is the only required method needs to be implemented in a class component.

lets create a div element which displays employee details like ID, Name , Location, Salary and return the div from this Method. To access the attributes that will be passed to this Component Class, in React we use this.props. Attribute Name. this.props contains the props that were defined by the caller of this component.



import React from 'react'; =>
This imports the React library, which is required to create React components.
import ReactDOM from 'react-dom'; =>

This imports the ReactDOM library, which is used to render React components into the actual DOM.



React JS: Fetch API Data Asynchronously

npx create-react-app fetch-api

Conceptual points to implement :

1. **Imports**:
   - The code imports `useState` and `useEffect` hooks from the React library. These hooks are used `for managing state` and `handling side effects` respectively.
2. **Component Function**:
   - `App()` function is defined, which represents a React component. This function will return JSX, which describes the structure of the UI.
3. **State Initialization**:
   - Two state variables are initialized using the `useState` hook:
     - `loading`: This state variable is used to indicate whether data is being fetched or not. It is initially set to *`true`*.

- `records`: This state variable holds the fetched data. It is initialized as an empty array *(`[]`)*.

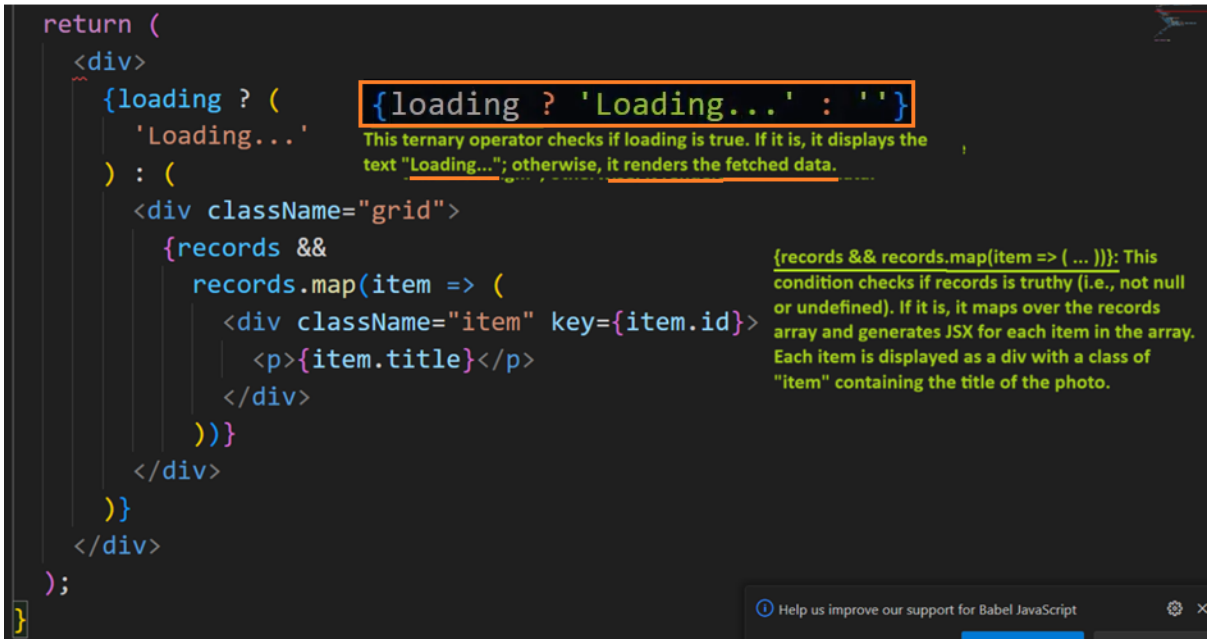
4. **Effect with Fetch Request**:
   - The `useEffect` hook is used to perform side effects in function components. In this case, it's fetching data from an API.
   - The `useEffect` hook runs once when the component mounts (due to the empty dependency array `[]`).
   - Inside the `useEffect`:
     - `setLoading(true)` is called to indicate that data fetching is in progress.
     - `fetch()` function is used to make a GET request to `https://jsonplaceholder.typicode.com/todos` API.
     - The response from the API is converted to JSON format using `.json()` method.
     - The data is then stored in the `records` state variable using `setRecords(data)`.
     - If there's an error during the fetch operation, it's caught and logged to the console.
     - Finally, `setLoading(false)` is called to indicate that data fetching is complete.
5. **Rendering Logic**:
   - The JSX returned by the `App()` function contains a conditional rendering logic:
     - If `loading` is `true`, it displays the text `'Loading...'`.
     - If `loading` is `false`, it renders the fetched data:
       - It maps over the `records` array and generates a `<div>` for each item with a unique `key` based on `item.id`.
       - Each `<div>` contains a `<p>` tag displaying the `title` of the item.

6. **Export**:
    - The `App` component is exported as the default export, making it available for use in other parts of the application.

In summary, this code sets up a React component (`App`) that fetches data from an API when it mounts, displays a loading message while fetching, and then renders the fetched data when it's available. It demonstrates the usage of state management with hooks (`useState`) and handling side effects (`useEffect`) in React functional components.

File Name: App.js






```
import React, { useState, useEffect } from 'react';
// This line imports React,
//along with two React hooks, useState and useEffect, which are
//used to manage state and perform side effects respectively.

function App() { // This defines a functional component named App.
  const [loading, setLoading] = useState(true);
  //useState hook to create a state variable called loading and a
function
  //to update it called setLoading. The initial value
  //of loading is true, indicating that data is currently being
loaded.
  const [records, setRecords] = useState([]);
  //creates a state variable called records to store the fetched
data,
  //initialized as an empty array, and a function setRecords to
update it.

  useEffect(() => { //useEffect hook, which is used to perform side
effects
    //in function components. Here, it's used to fetch data from the
JSONPlaceholder
```

```
    //API when the component mounts (since the dependency array []
is empty
    setLoading(true);
    fetch('https://jsonplaceholder.typicode.com/photos')
    //fetch('https://jsonplaceholder.typicode.com/todos')
      .then(response => response.json())
      .then(data => {
        setRecords(data);
        setLoading(false); // Move setLoading inside the fetch .then
block
      })
      .catch(error => console.error(error));
  }, []);

  return (
    <div>
      {loading ? (
        'Loading...'
      ) : (
        <div className="grid">
          {records &&
            records.map(item => (
              <div className="item" key={item.id}>
                <p>{item.title}</p>
              </div>
            ))}
        </div>
      )}
    </div>
  );
}

export default App;
```

Example-1:

defines a **MovieTicketBooking** component that allows users to select a movie and then select seats for that movie. It maintains the state of the selected movie and selected seats using React's **useState** hook. The available movies and their available seats are defined in the **movies** array. When a movie is selected, the available seats for that movie are displayed as buttons, and users can click on them to select seats. Selected seats are displayed below the seat selection area.

1. **Open VS Code**: Open Visual Studio Code on your computer.
2. **Create React Project**: If you haven't already set up a React project, you can do so by using Create React App. Open a terminal within VS Code, navigate to the directory where you want to create your project, and run the following command:

   npx create-react-app movie-ticket-booking
3. **Navigate to Project Directory**: Once the project is created, navigate to the project directory using the terminal:

   cd movie-ticket-booking
4. **Replace Code**: Inside VS Code, navigate to the **src** folder of your React project. Replace the contents of the **App.js** file with the code provided earlier. If you prefer, you can create a new component file (e.g., **MovieTicketBooking.js**) and put the code there.

```
import React, { useState } from 'react';

const MovieTicketBooking = () => {
 const [selectedMovie, setSelectedMovie] = useState(null);
 const [selectedSeats, setSelectedSeats] = useState([]);

 const movies = [
  { id: 1, title: 'Movie 1', availableSeats: 100 },
  { id: 2, title: 'Movie 2', availableSeats: 80 },
  { id: 3, title: 'Movie 3', availableSeats: 120 }
 ];

 const handleMovieSelect = (movie) => {
  setSelectedMovie(movie);
  setSelectedSeats([]);
 };
```

```
const handleSeatSelect = (seat) => {
  setSelectedSeats([...selectedSeats, seat]);
};

return (
  <div>
    <h2>Movie Ticket Booking</h2>
    <div>
      <h3>Select a movie:</h3>
      <ul>
        {movies.map(movie => (
          <li key={movie.id} onClick={() => handleMovieSelect(movie)}>
            {movie.title} ({movie.availableSeats} seats available)
          </li>
        ))}
      </ul>
    </div>
    {selectedMovie && (
      <div>
        <h3>Selected Movie: {selectedMovie.title}</h3>
        <h4>Select your seats:</h4>
        <div>
          {Array.from({ length: selectedMovie.availableSeats }, (_, index) => (
            <button
              key={index}
              disabled={selectedSeats.includes(index)}
              onClick={() => handleSeatSelect(index)}
            >
              {index + 1}
            </button>
          ))}
        </div>
        <div>
```

```
        <h4>Selected Seats:</h4>
        {selectedSeats.map(seat => (
          <span key={seat}>Seat {seat + 1}, </span>
        ))}
      </div>
      <button onClick={() => setSelectedSeats([])}>Reset</button>
    </div>
  )}
 </div>
 );
};

export default MovieTicketBooking;
```

5. **Run Project**: Save your changes, and start the development server by running:

   npm start

   This command will start the development server and open your default web browser to display the movie ticket booking application.

6. **View in Browser**: You can now view and interact with your movie ticket booking application in the web browser. Make sure your server is running, and then open **http://localhost:3000** in your browser.

